

55. "Fibers," *MSDN Library*, February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/fibers.asp>.
56. "Thread Pooling," *MSDN Library* February 2003 <msdn.microsoft.com/library/en-us/dllproc/base/thread_pooling.asp>.
57. "Queue User Worker Item," *MSDN Library*, February 2003, <msdn.microsoft.com/library/en-us/dllproc/base/queueuserworkitem.asp>.
58. Richter, J., "New Windows 2000 Pooling Functions Greatly Simplify Thread Management," *Microsoft Systems Journal*, April 1999, <www.microsoft.com/msj/0499/pooling/pooling.aspx>.
59. "Win_32 Thread," *MSDN Library*, July 2003, <msdn.microsoft.com/library/en-us/wmisdk/wmi/win32_thread.asp>.
60. Solomon, D., and M. Russinovich, *Inside Windows 2000*, 3d ed., Redmond: Microsoft Press, 2000, pp. 348-349.
61. Sun Microsystems, "Lightweight Processes," *SunOS System Service Overview*, Mountain View, CA: Sun Microsystems, December 1987, pp. 143-174.
62. Anderson, T. E.; B. N. Bershad; E. D. Lazowska; and H. M. Levy, "Scheduler Activations: Effective Kernel Support for User-Level Management of Parallelism," *ACM Transactions on Computer Systems*, Vol. 10, No. 1, February 1992, p. 54.
63. Benjamin, C, "The Fibers of Threads," *Compile Time (Linux Magazine)*, May 2001, <www.linux-mag.com/2001-05/compile_06.html>.
64. "Multithreading in the Solaris Operating Environment, A Technical White Paper," Sun Microsystems, 2002.
65. Blleloch, G. E., and G. J. Narlikar, "Pthreads for Dynamic and Irregular Parallelism," *Conference on High Performance Networking and Computing, Proceedings of the 1998 ACM/IEEE Conference on Supercomputing*, 1998, San Jose, CA.
66. Butenhof, David R., *Programming with POSIX Threads*, Boston, MA: Addison-Wesley, 1997: pp. 143-144.
67. Kleiman, S.; D. Shah; and B. Smaalders, *Programming with Threads*, Prentice Hall PTR, 1997.
68. Birrell, A. D, *An Introduction to Programming with Threads*. Technical Report 35, Digital Equipment Corporation, Systems Research Center, Palo Alto, CA, 1989.
69. Ousterhout, J., "Why threads are a bad idea (for most purposes)." In *USENIX Technical Conference (Invited Talk)*, Austin, TX, January 1996.

Do not put me to't, For I am nothing if not critical.

—William Shakespeare—

A person with one watch knows what time it is; a person with two watches is never sure.

—Proverb—

Bury leisure.

—Johann Elias Schlegel—

A really busy person never knows how much he weighs.

—Edgar Watson Howe—

Delays breed dangers.

—John Lyly—

By delaying he preserved the state.

—Quintus Ennius—

Chapter 5

Asynchronous Concurrent Execution

Objectives

After reading this chapter, you should understand:

- *the challenges of synchronizing concurrent processes and threads.*
- *critical sections and the need for mutual exclusion.*
- *how to implement mutual exclusion primitives in software.*
- *hardware mutual exclusion primitives.*
- *semaphore usage and implementation.*

5.1

Introduction

5.2

Mutual Exclusion

5.2.1 Java Multithreading Case Study,
Part II: A Producer/Consumer Relationship in
Java

5.2.2 Critical Sections

5.2.3 Mutual Exclusion Primitives

5.3

Implementing Mutual Exclusion Primitives

5.4

Software Solutions to the Mutual Exclusion Problem

5.4.1 Dekker's Algorithm

Anecdote: Why Should You Believe That
Your Software Is Functioning Correctly?

5.4.2 Peterson's Algorithm
5.4.3 N-Thread Mutual Exclusion:
Lamport's Bakery Algorithm
Biographical Note: Leslie Lamport

5.5

Hardware Solutions to the Mutual Exclusion Problem

5.5.1 Disabling Interrupts

5.5.2 Test-and-Set Instruction

5.5.3 Swap Instruction

5.6

Semaphores

Biographical Note: Edsger W. Dijkstra

5.6.1 Mutual Exclusion with Semaphores

5.6.2 Thread Synchronization with
Semaphores

5.6.3 Counting Semaphores

5.6.4 Implementing Semaphores

Anecdote: Unclear Requirements

Web Resources | Summary | Key Terms | Exercises | Recommended Reading | Works Cited

5.1 Introduction

In the previous two chapters, we introduced the concept of units of execution—processes and threads. Once again, we will focus our discussion on threads in this chapter, but most of what we say can apply to processes as well. If more than one thread exists in a system at the same time, then the threads are said to be **concurrent**.^{1, 2, 3} Two concurrent threads can execute completely independently of one another, or they can execute in cooperation. Threads that operate independently of one another but must occasionally communicate and synchronize to perform cooperative tasks are said to execute **asynchronously**.^{4, 5} Asynchronism is a complex topic—this chapter and Chapter 6, Concurrent Programming, discuss the organization and management of systems that support **asynchronous concurrent threads**.

We begin with a Java example that demonstrates the difficulty of writing correct programs even with as few as two asynchronous concurrent threads. If coded improperly, such programs can produce undesired and unpredictable results. In the remainder of the chapter, we discuss various mechanisms that programmers can use to create correct programs with asynchronous concurrent threads. In the next chapter, we present two classic asynchronism problems and implement their solutions in Java.

Self Review

1. (T/F) Communication and synchronization are necessary only for asynchronously executing threads.
2. (T/F) Threads that occasionally communicate, even though they normally operate independently of one another are said to execute synchronously.

Ans: 1) False. Communication and synchronization are necessary for asynchronously executing threads and processes. 2) False. Such threads are said to execute asynchronously.

5.2 Mutual Exclusion

Consider a mail server that processes e-mail for an organization. Suppose we want the system to continuously monitor the total number of e-mails that have been sent since the day began. Assume that the receipt of an e-mail is handled by one of several concurrent threads. Each time one of these threads receives an e-mail from a user, the thread increments a processwide shared variable, `mailCount`, by 1. Consider what happens if two threads attempt to increment `mailCount` simultaneously. First, assume that each thread runs the assembly-language code

```
LOAD mailCount
ADD 1
STORE mailCount
```

Assume that the `LOAD` instruction copies `mailCount` from memory to a register, the `ADD` instruction adds the immediate constant 1 from memory to the value in the register, and the `STORE` instruction copies the value in the register to memory. Suppose `mailCount` is currently 21687. Now suppose the first thread executes the `LOAD` and `ADD` instructions, thus leaving 21688 in the register (but not yet updating

the value in mailCount in memory, which is still 21687). Then, due to a quantum expiration, the first thread loses the processor and the system context switches to the second thread. The second thread now executes all three instructions, thus setting mailCount to 21688. This thread loses the processor, and the system context switches back to the first thread, which then continues by executing the STORE instruction—also placing 21688 into mailCount. Due to the uncontrolled access to the shared variable mailCount, the system has essentially lost track of one of the e-mails—mailCount should be 21689. In the case of an e-mail management application, such an error may seem minor. A similar error occurring in a mission-critical application such as air traffic control could cost lives.

The cause of this incorrect result is the writing of the shared variable mailCount. Clearly, many concurrent threads may read data simultaneously without this difficulty. But when one thread reads data that another thread is writing, or when one thread writes data that another thread is also writing, indeterminate results can occur.^{6,7,8}

We can solve this problem by granting each thread **exclusive** access to mailCount. While one thread increments the shared variable, all other threads desiring to do so will be made to wait. When the executing thread finishes accessing the shared variable, the system will allow one of the waiting processes to proceed. This is called **serializing** access to the shared variable. In this manner, threads will not be able to access shared data simultaneously. As each thread proceeds to update the shared variable, all others are excluded from doing so simultaneously. This is called **mutual exclusion**.^{9, 10, 11} As we will see in this and subsequent chapters, waiting threads must be carefully managed to ensure that they will be able to proceed within a "reasonable" amount of time.

Self Review

1. In the mailCount example above, would it be acceptable if several threads simultaneously read the value without updating it?
2. (T/F) Suppose each of two threads that share a variable needs to update that shared variable at some point during the thread's execution. If the threads are not made to mutually exclude each other from updating the shared variable simultaneously, then on any execution of two threads, the system will fail.

Ans: 1) Yes, mutual exclusion is necessary only when threads update the shared variable. 2) False. It is possible that the threads will attempt to update the shared variable at different times, and the program could thus function correctly.

5.2.1 Java Multithreading Case Study, Part II: A Producer/Consumer Relationship in Java

In a **producer/consumer relationship**, the **producer** portion of the application generates data and stores it in a shared object, and the **consumer** portion reads data from the shared object. One example of a common producer/consumer relationship is print spooling. A word processor spools data to a buffer (typically a file) and that data is subsequently consumed by the printer as it prints the document. Similarly, an applica-

tion that copies data onto compact discs places data in a fixed-size buffer that is emptied as the CD-RW drive burns the data onto the compact disk.

Consider a multithreaded producer/consumer relationship implemented in Java in which a **producer thread** generates data and places it into a buffer capable of holding a single value and a **consumer thread** reads data from the buffer. If the producer waiting to put the next data into the buffer determines that the consumer has not yet read the previous data from the buffer, the producer should call `wait` so the consumer gets a chance to read unconsumed data before it is overwritten. When the consumer reads the data, it should call `notify` to allow the (possibly waiting) producer to store the next value. Object method `notify` transitions a thread from the *waiting* state to the *ready* state. If the consumer finds the buffer empty (because the producer has not yet produced its first data) or finds that the previous data has already been read, the consumer should call `wait` to place itself in the *waiting* state; otherwise, the consumer might read "garbage" from an empty buffer or might erroneously process the same data again. When the producer places the next data into the buffer, the producer should call `notify` to allow the (possibly waiting) consumer thread to proceed, so the consumer can read the new data. Note that `notify` has no effect when none of an application's threads are waiting.

Let us implement this example in Java to see how logic errors can arise if we do not synchronize access among multiple threads manipulating shared data. The example that follows (Fig. 5.1-Fig. 5.5) implements a producer/consumer relationship in which a producer thread sequentially writes numbers (1 to 4) into a shared buffer—a memory location shared between two threads (a single `int` variable called `buffer` in Fig. 5.4). The consumer thread reads this data from the shared buffer and displays the data. The program's output (three samples are shown in Fig. 5.5) shows the values that the producer writes (produces) into the shared buffer and the values that the consumer reads (consumes) from the shared buffer.

Each value the producer writes to the shared buffer must be consumed exactly once (and in order) by the consumer thread. However, the threads in this example are not **synchronized**, meaning that they do not cooperate when performing their tasks (which, as we have seen, is especially important when one of the threads is writing to the buffer). Therefore, data (possibly multiple values) can be lost if the producer places new data into the shared buffer before the consumer consumes the previous data, and data can be incorrectly duplicated (possibly many times) if the consumer consumes data again before the producer produces the next value. The consequence of not synchronizing access to shared data is somewhat like the consequence of not having a traffic light at a busy intersection.

To show these possibilities, the consumer in the following example keeps a total of all the values it reads. The producer produces values from 1 to 4 in order. In the traditional producer/consumer relationship, the consumer reads each produced value once and only once. In addition, the consumer cannot read each value until after the producer has produced it. Therefore, the producer must always produce a value before the consumer reads that value, and the total of the values consumed in

our example must be 10. However, if you execute this program several times (see the sample outputs of Fig. 5.5), you will see that the total is rarely, if ever, 10. The producer and consumer threads in the example each sleep for random intervals of up to three seconds between performing their tasks, simulating some lengthy delay such as waiting for user input or for some event to occur. Thus, we do not know exactly when the producer will attempt to write a new value, nor do we know when the consumer will attempt to read a value.

The Java program consists of interface `Buffer` (Fig. 5.1) and four classes—`Producer` (Fig. 5.2), `Consumer` (Fig. 5.3), `UnsynchronizedBuffer` (Fig. 5.4) and `SharedBufferTest` (Fig. 5.5). Interface `Buffer` declares methods `set` and `get` that a `Buffer` must implement to enable a `Producer` thread to place a value in the `Buffer` and enable a `Consumer` thread to retrieve a value from the `Buffer`, respectively. This interface is implemented in Fig. 5.4 (line 4).

Class `Producer` (Fig. 5.2)—a subclass of `Thread` (line 5)—contains field `sharedLocation` (line 7), a constructor (lines 9-14) and a `run` method (lines 18-40). The constructor initializes `Buffer` reference `sharedLocation` (line 13) with a reference to an object that implements the `Buffer` interface. That object is created in `main` (Fig. 5.5; lines 6-18) and passed to the constructor as the parameter `shared` (Fig. 5.2; line 10); the constructor initializes the `Buffer` reference `sharedLocation` (line 13) to be a reference to the parameter `shared`. The producer thread in this program executes the tasks specified in method `run` (lines 18-40). The `for` statement in lines 20-35 loops four times. Each iteration of the loop invokes `Thread` method `sleep` (line 25) to place the producer thread into the sleeping state for a random time interval between 0 and 3 seconds (to simulate a lengthy operation).

[Note: Normally, a thread awakens when its sleep time expires. A sleeping thread can be awakened early if another thread calls the sleeping thread's `interrupt` method. If this occurs, `sleep` "throws" an exception (of type `InterruptedException`) to indicate that the thread was interrupted before its sleep time expired. In Java, this exception must be "handled," which requires the `sleep` method call to appear in a `try` block that is followed by a catch handler. The `try` block contains the code that might throw an exception. The catch handler specifies the type of exception it handles. In this example, the catch handler prints a stack trace, then the program continues with the next statement after the `try...catch` sequence.]

```

1 //Fig. 5.1: Buffer.Java
2 // Buffer interface specifies methods to access buffer data.
3
4 public interface Buffer
5 {
6     public void set( int value ); // place value into Buffer
7     public int get();           // return value from Buffer
8 }

```

Figure 5.1 | *Buffer interface used in producer/consumer examples.*

When the thread awakens, line 26 passes the value of control variable `count` to the `Buffer` object's `set` method to change the shared buffer's value. When the loop completes, lines 37-38 display a message in the console window indicating that the thread finished producing data and that the thread is terminating. Next, method `run` terminates (line 40) and the producer thread enters the *dead* state. It is important to note that any method called from a thread's `run` method (such as `Buffer` method `set` in line 26) executes as part of that thread of execution. In fact, each thread has its own method-call stack.

```

1  // Fig. 5,2: Producer.Java
2  // Producer's run method controls a producer thread that
3  // stores values from 1 to 4 in Buffer sharedLocation.
4
5  public class Producer extends Thread
6  {
7      private Buffer sharedLocation; // reference to shared object
8
9      // Producer constructor
10 public Producer( Buffer shared )
11 {
12     super( "producer" ); // create thread named "Producer"
13     sharedLocation = shared; // initialize sharedLocation
14 } // end Producer constructor
15
16 // Producer method run stores values from
17 // 1 to 4 in Buffer sharedLocation
18 public void run()
19 {
20     for ( int count = 1; count <= 4; count++ )
21     {
22         // sleep 0 to 3 seconds, then place value in Buffer
23         try
24         {
25             Thread.sleep( ( int ) ( Math.random() * 3001 ) );
26             sharedLocation.set( count ); // write to the buffer
27         } // end try
28
29         //if sleeping thread interrupted, print stack trace
30         catch ( InterruptedException exception )
31         {
32             exception.printStackTrace();
33         } // end catch
34     }
35 } // end for

```

Figurv 5.2 | *Producer class represents the producer thread in a producer/consumer relationship. (Part 1 of 2.)*

```

36
37     System.err.println( getName0 + " done producing." +
38         "\nTerminating " + getName0 + "." );
39
40 } // end method run
41
42 } // end class Producer

```

Figure 5.2 | Producer class represents the producer thread in a producer/consumer relationship. (Part 2 of 2.)

Class Consumer (Fig.5.3) contains instance variable `sharedLocation` (line 7), a constructor (lines 9-14) and a run method (lines 16-41). The constructor initializes Buffer reference `sharedLocation` (line 13) with a reference to an object that implements the Buffer interface. That object is created in main (Fig.5.5) and passed to the constructor as the parameter `shared` (Fig. 5.3; line 10). As we will see (Fig. 5.5; lines 12-13), this is the same `UnsyncronizedBuffer` object that is used to initialize the Producer object; thus, the producer and consumer threads share the object. The consumer thread in this program performs the tasks specified in method `run` (Fig. 5.3; lines 16-41). The loop in lines 22-36 loops four times. Each iteration of the loop invokes Thread method `sleep` (line 27) to put the consumer thread into the sleeping state for a random time interval between 0 and 3 seconds. Next, line 28 uses the Buffer's `get` method to retrieve the value in the shared buffer, then adds the value to variable `sum`. When the loop completes, lines 38-39 display a line in the console window indicating the sum of the consumed values and the fact that the consumer thread is terminating. Then, method `run` terminates (line 41), and the consumer thread enters the *dead* state.

```

1 //Fig. 5.3: Consumer.Java
2 // Consumer's run method controls a thread that loops four
3 // times and reads a value from sharedLocation each time.
4
5 public class Consumer extends Thread
6 {
7     private Buffer sharedLocation; // reference to shared object
8
9     // Consumer constructor
10    public Consumer( Buffer shared )
11    {
12        super( "Consumer" ); // create thread named "Consumer"
13        sharedLocation = shared; // initialize sharedLocation
14    } // end Consumer constructor

```

Figure 5.3 | Consumer class represents the consumer thread in a producer/consumer relationship. (Part 1 of 2.)

```

15
16 // read sharedLocation's value four times and sum the values
17 public void run()
18 {
19     int sum = 0;
20
21     // alternate between sleeping and getting Buffer value
22     for (int count = 1; count <= 4; ++count )
23     {
24         // sleep 0-3 seconds, read Buffer value and add to sum
25         try
26         {
27             Thread.sleep( ( int ) ( Math.random() * 3001 ) );
28             sum += sharedLocation.get();
29         }
30
31         //if sleeping thread interrupted, print stack trace
32         catch ( InterruptedException exception )
33         {
34             exception.printStackTrace();
35         }
36     } // end for
37
38     System.err.println( getName() + " read values totaling: "
39         + sum + "AnTerminating " + getName() + "." );
40
41 } // end method run
42
43 } // end class Consumer

```

Figure 5.3 | *Consumer class represents the consumer thread in a producer/consumer relationship. (Part 2 of 2.)*

[Note: We use a randomly generated interval with method `sleep` in method `run` of both the `Producer` and `Consumer` classes to emphasize the fact that, in multithreaded applications, it is unclear when and for how long each thread will perform its task. Normally, these thread-scheduling issues are the job of the operating system. In this program, our thread's tasks are quite simple—for the producer, loop four times, each time setting a value in the shared buffer; for the consumer, loop four times, each time retrieving a value from the shared buffer and adding the value to variable `sum`. Without the `sleep` method call, and if the producer were to execute first, it most likely would complete its task before the consumer got a chance to execute; if the consumer were to execute first, most likely it would consume the "null" value (-1, specified in `UnsynchronizedBuffer`, line 6 of Fig. 5.4) four times, then terminate (displaying the invalid sum, -4) before the producer produced even its first value.]

Class `UnsynchronizedBuffer` (Fig. 5.4) implements interface `Buffer` (line 4) defined in Fig. 5.1 and declares the variable `buffer` (line 6), shared between the Producer and Consumer. Variable `buffer` is initialized with the value `-1`. This value is used to demonstrate the case in which the Consumer attempts to consume a value before the Producer places a value in `buffer`. Methods `set` (lines 8-15) and `get` (lines 17-24) do not synchronize access to field `buffer` (we will see how to do this shortly). Method `set` simply assigns its parameter to `buffer` (line 14) and method `get` simply returns the value of `buffer` (line 23). Note that each method uses class `Thread`'s static method `currentThread` to obtain a reference to the currently executing thread, then uses that thread's method `getName` to obtain the thread's name for output purposes (lines 11 and 20).

Class `SharedBufferTest` (Fig. 5.5) contains method `main` (lines 6-18), which launches the application. Line 9 instantiates a shared `UnsynchronizedBuffer` object and assigns it to `Buffer` reference `sharedLocation`. This object stores the data that will be shared between the producer and consumer threads. Lines 12-13 create the Producer object `producer` and the Consumer object `consumer`. Each of

```

1 // Fig. 5.4: UnsynchronizedBuffer.java
2 // UnsynchronizedBuffer represents a single shared integer.
3
4 public class UnsynchronizedBuffer implements Buffer
5 {
6     private int buffer = -1; // shared by Producer and Consumer
7
8     // place value into buffer
9     public void set( int value )
10    {
11        System.err.println( Thread.currentThread().getName() +
12            " writes " + value );
13
14        buffer = value;
15    } // end method set
16
17    // return value from buffer
18    public int get()
19    {
20        System.err.println( Thread.currentThread().getName() +
21            " reads " + buffer );
22
23        return buffer;
24    } // end method get
25
26 } // end class UnsynchronizedBuffer

```

Figure 5.4 | `UnsynchronizedBuffer` class maintains the shared integer that is accessed by a producer thread and a consumer thread via methods `set` and `get`

the constructor calls in these lines passes `sharedLocation` as the argument to the constructor (lines 9-14 of Fig. 5.2 for Producer and lines 9-14 of Fig. 5.3 for Consumer), so that each thread accesses the same Buffer. Next, lines 15-16 of Fig. 5.5 invoke method `start` on the producer and consumer threads, respectively, to place them in the *ready* state. This launches the execution of these threads by making an implicit call to each thread's `run` method. A thread's `run` method will be called when the thread is dispatched to a processor for the first time. After the producer and consumer threads are started, method `main` (i.e., the main thread of execution) terminates and the main thread enters the *dead* state. Once both the producer and consumer threads enter the *dead* state (which occurs when their `run` methods terminate), the program terminates.

Recall from the overview of this example that the Producer thread must produce a value before the Consumer thread consumes a value, and every value produced by the producer must be consumed exactly once by the consumer. However, when we study the first output of Fig. 5.5, we see that the consumer retrieved the value -1 before the producer ever placed a value in the buffer. The first value produced (1) was consumed three times. Furthermore, the consumer finished executing before the producer had an opportunity to produce the values 2, 3 and 4, so those three values were lost. An incorrect total of 2 was produced.

In the second output, we see that the value 1 was lost, because the values 1 and 2 were produced before the consumer thread could read the value 1 (so the

```

1  // Fig. 5.5: SharedBufferTest.java
2  // SharedBufferTest creates producer and consumer threads,
3
4  public class SharedBufferTest
5  {
6      public static void main( String [ ] args )
7      {
8          // create shared object used by threads
9          Buffer sharedLocation = new UnsynchronizedBuffer();
10
11         // create producer and consumer objects
12         Producer producer = new Producer( sharedLocation );
13         Consumer consumer = new Consumer( sharedLocation );
14
15         producer.start(); // start producer thread
16         consumer.start(); // start consumer thread
17
18     } // end main
19
20 } // end class SharedCell

```

Figure 5.5 | Shared Buffer class enables threads to modify a shared object Without synchronization. (Part 1 of 2.)

Sample Output 1.

```
Consumer reads -1
Producer writes 1
Consumer reads 1
Consumer reads 1
Consumer reads 1
Consumer read values totaling: 2.
Terminating Consumer.
Producer writes 2
Producer writes 3
Producer writes 4
Producer done producing.
Terminating Producer.
```

Sample Output 2:

```
Producer writes 1
Producer writes 2
Consumer reads 2
Producer writes 3
Consumer reads 3
Producer writes 4
Producer done producing.
Terminating Producer.
Consumer reads 4
Consumer reads 4
Consumer read values totaling: 13.
Terminating Consumer.
```

Sample Output 3:

```
Producer writes 1
Consumer reads 1
Producer writes 2
Consumer reads 2
Producer writes 3
Consumer reads 3
Producer writes 4
Producer done producing.
Terminating Producer.
Consumer reads 4
Consumer read values totaling: 10.
Terminating Consumer.
```

Figure 5.5 | Shared Buffer class enables threads to modify a shared object without synchronization. (Part 2 of 2.)

value 2 overwrote the value 1 in the buffer). Also, the value 4 was consumed twice. Once again, an incorrect total was produced. The last sample output demonstrates that it is possible, with some luck, to get a proper output; clearly, this behavior cannot be guaranteed.

This example demonstrates that access to a shared object by concurrent threads must be controlled carefully; otherwise, a program may produce incorrect results. We provide a solution to this problem in the next chapter. The remainder of this chapter discusses how threads control concurrent access to shared data by enforcing mutually exclusive access to that data.

Self Review

1. (T/F) In the producer/consumer example shown in Fig. 5.1-Fig. 5.5, the output would be correct and consistent if the calls to sleep were removed.
2. What are the smallest and largest possible values of sum at the end of the execution of the example shown in Fig. 5.1-Fig. 5.5, and how would those values be achieved?

Ans: 1) False. The sleep calls were put there simply to emphasize the fact that there is uncertainty within a system regarding the order and timing in which instructions are executed. Still, we cannot predict the speeds at which these threads will advance, so the outputs could still be incorrect. 2) The smallest value would occur if the consumer tried consuming four values before the producer produced any. In this case, sum would be -4. The largest value would occur if the producer produced four values before the consumer consumed any. In this case, the consumer would consume the value four times, and sum would be 16.

5.2.2 Critical Sections

Mutual exclusion needs to be enforced only when threads access shared modifiable data. When threads are performing operations that do not conflict with one another (e.g., reading variables), the system should allow the threads to proceed concurrently. When a thread is accessing shared modifiable data, it is said to be in a critical section (or critical region).¹² To prevent the kinds of errors we encountered earlier, the system should ensure that only one thread at a time can execute the instructions in its critical section for a particular resource. If one thread attempts to enter its critical section while another is in its own critical section, the first thread should wait until the executing thread exits the critical section. Once a thread has exited its critical section, a waiting thread (or one of the waiting threads, if there are several), may enter and execute its critical section. Any thread that does not need to enter its critical section may execute regardless of whether a critical section is occupied.

A system that enforces mutual exclusion must carefully control access to, and execution in, critical sections. A thread in a critical section has exclusive access to shared, modifiable data, and all other threads currently requiring access to that data are kept waiting. Therefore, a thread should execute a critical section as quickly as possible. A thread must not block inside its critical section, and critical sections must be carefully coded to avoid—for example—the possibility of infinite loops. If a thread in a critical section terminates, either voluntarily or involuntarily, then the

operating system, in performing its **termination housekeeping**, must release mutual exclusion so that other threads may enter their critical sections.

Enforcing mutually exclusive access to critical sections is one of the key problems in concurrent programming. Many solutions have been devised: some software solutions and some hardware solutions; some rather low-level and some high-level; some requiring voluntary cooperation among threads and some demanding rigid adherence to strict protocols. We will examine a variety of these solutions in the upcoming sections.

Self Review

1. Why is it important for a thread to execute a critical section as quickly as possible?
2. What might happen if the operating system did not perform termination housekeeping?

Ans: 1) If other threads are waiting to execute their critical sections, they will be delayed from doing so while any thread is in its critical section. If critical sections do not execute quickly, overall system performance could suffer. 2) A thread could terminate while in its critical section, and never release mutual exclusion. The threads waiting to enter their critical sections would never be able to enter. As we will see in Chapter 7, Deadlock and Indefinite Postponement, these threads will deadlock.

5.2.3 Mutual Exclusion Primitives

The following pseudocode properly describes the e-mail counting mechanism of Section 5.2, Mutual Exclusion. Notice that we use the words `enterMutualExclusion()` and `exitMutualExclusion()`. These words are constructs that encapsulate the thread's critical section—when a thread wants to enter its critical section, the thread must first execute `enterMutualExclusion()`; when a thread exits the critical section, it executes `exitMutualExclusion()`. Because these constructs invoke the most fundamental operations inherent to mutual exclusion, they are sometimes called mutual exclusion primitives.

```
while (true) {
    Receive e-mail           // executing outside critical section
    enterMutualExclusion()    // want to enter critical section
    Increment mailCount      // executing inside critical section
    exitMutualExclusion()     // leaving critical section
}
```

Let us consider how these primitives can provide mutual exclusion. For simplicity in the example presented in this and the next several sections, we shall assume only two concurrent threads and one processor. Handling n concurrent processes or threads is a considerably more complex topic, which we discuss in Section 5.4.3, N-Thread Mutual Exclusion: Lamport's Bakery Algorithm. We will also discuss how mutual exclusion is enforced on multiprocessor systems.

Assume that threads T_1 and T_2 of the same process are both executing in the system. Each thread manages e-mail messages, and each thread contains instructions that correspond to the preceding pseudocode. When T_1 reaches the `enterMutual Ex-`

clusion() line, the system must determine whether T_2 is already in its critical section. If T_2 is not in its critical section, then T_1 enters its critical section, increments shared variable mail Count and executes exitMutualExclusion() to indicate that T_1 has left its critical section. If, on the other hand, when T_1 executes enterMutualExclusion(), T_2 is in its critical section, then T_1 must wait until T_2 executes exitMutualExclusion(), at which point T_1 enters its critical section. If T_1 and T_2 simultaneously execute enterMutualExclusion(), then only one of the threads will be allowed to proceed, and one will be kept waiting. For the moment, we shall assume that the system randomly selects the thread that will proceed. [As we will see, such a policy could lead to indefinite postponement of one of the threads if the other is always selected when the threads repeatedly try to enter their critical sections.] In the next several sections, we discuss various mechanisms for implementing the enterMutualExclusion() and exitMutualExclusion() mutual exclusion primitives.

Self Review

1. What will happen if a thread does not call enterMutualExclusionO before accessing shared variables in a critical section?
2. What will happen if a thread that is in its critical section does not call exitMutualExclusionO?

Ans: 1) If other threads also enter their critical section, there could be indeterminate results that could have serious consequences. 2) All other threads that are waiting to enter their critical sections would never be allowed to do so. As we will see in Chapter 7, Deadlock and Indefinite Postponement, all of these other threads would eventually deadlock.

5.3 Implementing Mutual Exclusion Primitives

Each initial mutual exclusion solutions that we discuss provides an implementation of enterMutualExclusion() and exitMutualExclusion() that exhibits the following properties:

1. The solution is implemented purely in software on a machine without specially designed mutual exclusion machine-language instructions. Each machine-language instruction is executed **indivisibly**—i.e., once started, it completes without interruption. If a system contains multiple processors, several threads could try to access the same data item simultaneously. As we will see, many mutual exclusion solutions for uniprocessor systems rely on access to shared data, meaning that they might not work on multiprocessor systems (see Section 15.9, Multiprocessor Mutual Exclusion). For simplicity, we will assume that the system contains one processor. Again, we will deal with multiprocessor systems later.
2. No assumption can be made about the relative speeds of asynchronous concurrent threads. This means that any solution must assume that a thread can be preempted or resumed at any time during its execution and that the rate of execution of each thread may not be constant or predictable.

3. A thread that is executing instructions outside its critical section cannot prevent any other threads from entering their critical sections.
4. A thread must not be indefinitely postponed from entering its critical section. [Note: Once inside an improperly coded critical section, a thread could certainly "misbehave" in ways that could lead to indefinite postponement or even deadlock. We discuss these issues in depth in Chapter 7, *Deadlock and Indefinite Postponement*.]

Self Review

1. Why do you suppose that we cannot predict the relative speeds of asynchronous concurrent threads?
2. How can an improperly coded thread cause indefinite postponement or deadlock?

Ans: 1) One reason is that hardware interrupts, which may be randomly distributed, can disrupt thread execution at any time. Over any given period, these interrupts may allow some threads to execute longer than others. In addition, biases in resource-scheduling algorithms may favor certain threads. **2)** A thread that never left its critical section (e.g., by entering an infinite loop) would eventually indefinitely postpone or deadlock all other threads as they tried to enter their critical sections.

5.4 Software Solutions to the Mutual Exclusion Problem

An elegant software implementation of mutual exclusion was first presented by the Dutch mathematician Dekker. In the next section we follow Dijkstra's development of **Dekker's Algorithm**, an implementation of mutual exclusion for two threads.¹³ The arguments presented introduce many of the subtleties in concurrent programming that make this such an interesting field of study. Then, we discuss simpler and more efficient algorithms developed by G. L. Peterson¹⁴ and L. Lamport.¹⁵

5.4.1 Dekker's Algorithm

In this section we examine several attempts to implement mutual exclusion. Each implementation contains a problem that the subsequent one overcomes. The section culminates in the presentation of a correct software implementation to mutual exclusion that is free of deadlock and indefinite postponement.

First Version (Introducing lockstep Synchronization and Busy Waiting)

Figure 5.6 shows a first effort at specifying the code for enforcing mutual exclusion between two threads. The pseudocode is presented using a C-like syntax. Each thread's instructions can be broken into three parts: noncritical instructions (i.e., instructions that do not modify shared data), critical instructions (i.e., instructions that do modify shared data) and the instructions that ensure mutual exclusion (i.e., instructions that implement `enterMutualExclusion()` and `exitMutualExclusion()`). Each thread repeatedly enters and exits its critical section until it is done.

Under this version of mutual exclusion, the system uses a variable called `threadNumber`, to which both threads have access. Before the system starts execut-

```

1 System:
2
3 int threadNumber = 1;
4
5 startThreads(); // initialize and launch both threads
6
7 Thread T1:
8
9 void main() {
10
11     while ( !done )
12     {
13         while ( threadNumber == 2 ); // enterMutualExclusion
14
15         // critical section code
16
17         threadNumber = 2; // exitMutualExclusion
18
19         // code outside critical section
20
21     } // end outer while
22
23 } // end Thread T1
24
25 Thread T2:
26
27 void main() {
28
29     while ( !done )
30     {
31         while ( threadNumber == 1 ); // enterMutualExclusion
32
33         // critical section code
34
35         threadNumber = 1; // exitMutualExclusion
36
37         // code outside critical section
38
39     } // end outer while
40
41 } // end Thread T2

```

Figure 5.6 | Mutual exclusion implementation- version 1.

ing the threads, threadNumber is set to 1 (line 3). Then the system starts both threads. The enterMutualExclusion() primitive is implemented as a single while loop that loops indefinitely until variable threadNumber becomes equal to the number of the thread (see lines 13 and 31 in threads T₁ and T₂, respectively). The

`exitMutualExclusion()` primitive is implemented as a single instruction that sets `threadNumber` to the number of the other thread (see lines 17 and 35 in threads T_1 and T_2 , respectively).

Consider one way in which execution might proceed under this implementation. Assume T_1 begins executing first. The thread executes the while loop (line 13) that acts as `enterMutualExclusion()`. Because `threadNumber` is initially 1, T_1 enters its critical section (the code indicated by the comment in line 15). Now assume that the system suspends T_1 and begins executing T_2 . The second thread finds `threadNumber` equal to 1 and remains "locked" in the while loop (line 31). This guarantees mutual exclusion, because T_2 cannot enter its critical section until T_1 exits its critical section and sets `threadNumber` to 2 (line 17).

Eventually, T_1 finishes executing in its critical section (recall that we are assuming that the critical sections contain no infinite loops and that threads do not die or block inside critical sections). At this point, T_1 sets `threadNumber` to 2 (line 17) and continues to its "noncritical" instructions. Now T_2 is free to enter its critical section.

Although this implementation guarantees mutual exclusion, the solution has significant drawbacks. In the `enterMutualExclusion()` primitive, the thread uses the processor to perform essentially no work (i.e., the thread repeatedly tests the value of `threadNumber`). Such a thread is said to be **busy waiting**. Busy waiting can be an ineffective technique for implementing mutual exclusion on uniprocessor systems. Recall that one goal of multiprogramming is to increase processor utilization. If our mutual exclusion primitive uses processor cycles to perform work that is not essential to the thread, then processor time is wasted. This overhead, however, is limited to short periods of time (i.e., when there is contention between the threads to enter their critical sections).

A more damaging drawback of this implementation is that it violates one of the key constraints we identified in Section 5.3, Implementing Mutual Exclusion Primitives, namely that a thread that is not in its critical section should not affect a thread that desires to enter its own critical section. For example, T_1 must enter the critical section first, because the system initially sets `threadNumber` to 1. If T_2 tries to go first, it begins execution and unsuccessfully attempts to enter its critical section. Eventually, T_1 executes, entering its critical section and setting `threadNumber` to 2. Thus, T_2 may be delayed considerably before it may enter its critical section. In fact, the threads must enter and leave their critical sections in strict alternation. If one thread needs to enter its critical section more frequently than the other, the faster thread will be constrained to operate at the speed of the slower thread. This is called the **lockstep synchronization** problem.* The next mutual exclusion implementation eliminates this problem but introduces another.

Of course, the problem in Section 5.2.1 required lockstep synchronization because the buffer was of unit size. However, almost all buffers contain several entries, meaning that lockstep synchronization causes inefficient behavior if the producer and consumer operate at different speeds.

Second Version (Violating Mutual Exclusion)

Figure 5.7 contains a solution that attempts to eliminate the lockstep synchronization of the previous implementation. In the first solution, the system maintained only a single global variable—this forced the lockstep synchronization. Version 2 maintains two variables —`t1Inside` that is true if T_1 is inside its critical section, and `t2Inside` that is true if T_2 is inside its critical section.

```

1  System:
2
3  boolean t1Inside = false;
4  boolean t2Inside = false;
5
6  startThreads(); // initialize and launch both threads
7
8  Thread T1:
9
10 void main() {
11
12     while ( !done )
13     {
14         while ( t2Inside ); // enterMutualExclusion
15
16         t1Inside = true; // enterMutualExclusion
17
18         // critical section code
19
20         t1Inside = false; // exitMutualExclusion
21
22         // code outside critical section
23
24     } // end outer while
25
26 } // end Thread T1
27
28 Thread T2:
29
30 void main() {
31
32     while ( !done )
33     {
34         while ( t1Inside ); // enterMutualExclusion
35
36         t2Inside = true; // enterMutualExclusion
37
38         // critical section code

```

Figure 5.7 | Mutual exclusion implementation- version 2. (Part 1 of 2.)

```

39
40     t2Inside = false; // exitMutualExclusion
41
42     // code outside critical section
43
44 } // end outer while
45
46 } // end Thread T2

```

Figure 5.7 | Mutual exclusion implementation- version 2. (Part 2 of 2)

By using two variables to govern access to the critical sections (one for each thread), we are able to eliminate lockstep synchronization in our second mutual exclusion implementation. T_1 will be able to continuously enter its critical section as many times as necessary while `t2Inside` is false. Also, if T_1 enters the critical section and sets `t1Inside` to true, then T_2 will busy wait. Eventually, T_1 will finish its critical section and perform its mutual exclusion exit code, setting `t1Inside` to false.

Although this solution eliminates the lockstep synchronization issue, it unfortunately does not guarantee mutual exclusion. Consider the following situation. Variables `t1Inside` and `t2Inside` are both false, and both threads attempt to enter their critical sections at the same time. T_1 tests the value of `t2Inside` (line 14), determines that the value is false and proceeds to the next statement. Now suppose that the system preempts T_1 before it can execute line 16 (which it needs to do to keep T_2 out of its own critical section). T_2 now executes, determines that `t1Inside` is false and so T_2 enters its critical section. While T_2 is in its critical section, T_2 is preempted and T_1 resumes execution in line 16, setting `t1Inside` to true and entering its critical section. Both threads are concurrently executing their critical sections, violating mutual exclusion.

Third Version (Introducing Deadlock)

Version 2 failed because between the time a thread determines in its while test that it can go ahead (lines 14 and 34) and the time the thread sets a flag to say that it is in its critical section (lines 16 and 36), there is the possibility that the other thread could gain control, pass its while test and enter its critical section. To correct this problem, once a thread attempts the while test, it must be assured that the other process cannot proceed past its own while test. Version 3 (Fig. 5.8) attempts to resolve this problem by having each thread set its own flag prior to entering the while loop. Thus, T_1 indicates its desire to enter its critical section by setting `t1WantsToEnter` to true. If `t2WantsToEnter` is false, then T_1 enters its critical section and prevents T_2 from entering its own critical section. Thus mutual exclusion is guaranteed, and it seems that we have a correct solution. We ensure that, once a thread signals its intent to enter the critical section, another thread cannot enter the critical section.

```

1  System:
2
3  boolean t1WantsToEnter = false;
4  boolean t2WantsToEnter = false;
5
6  startThreads(); // initialize and launch both threads
7
8  Thread T1:
9
10 void main()
11 {
12     while ( !done )
13     {
14         t1WantsToEnter = true; // enterMutuaiExclusion
15
16         while ( t2WantsToEnter ); // enterMutualExclusion
17
18         // critical section code
19
20         t1WantsToEnter = false; // exitMutualExclusion
21
22         // code outside critical section
23
24     } // end outer while
25
26 } // end Thread T1
27
28 Thread T2:
29
30 void main()
31 {
32     while ( !done )
33     {
34         t2WantsToEnter = true; // enterMutualExclusion
35
36         while ( t1WantsToEnter ); // enterMutualExclusion
37
38         // critical section code
39
40         t2WantsToEnter = false; // exitMutualExclusion
41
42         // code outside critical section
43
44     } // end outer while
45
46 } // end Thread T2

```

Figure 5.8 | Mutual exclusion implmentation- version 3.

One problem has been solved, but another has been introduced. If each thread sets its flag before proceeding to the while test, then each thread will find the other's flag set and will loop forever in the while. This is an example of a two-process **deadlock**, which is discussed in depth in Chapter 7, Deadlock and Indefinite Postponement.

Version Fourth (Introducing Indefinite Postponement)

To create an effective mutual exclusion implementation, we need a way to "break out" of the infinite loops we encountered in the previous version. Version 4 (Fig. 5.9) accomplishes this by forcing each looping thread to repeatedly set its flag to false for brief periods. This allows the other thread to proceed through its while loop with its own flag set to true.

```

1  System:
2
3  boolean t1WantsToEnter = false;
4  boolean t2WantsToEnter = false;
5
6  startThreads(); // initialize and launch both threads
7
8  Thread T1:
9
10 void main()
11 {
12     while ( !done )
13     {
14         t1WantsToEnter = true; // enterMutualExclusion
15
16         while ( t2WantsToEnter ) // enterMutualExclusion
17         {
18             t1WantsToEnter = false; // enterMutualExclusion
19
20             // wait for small, random amount of time
21
22             t1WantsToEnter = true;
23         } // end while
24
25         // critical section code
26
27         t1WantsToEnter = false; // exitMutualExclusion
28
29         // code outside critical section
30
31     } // end outer while
32

```

Figure 5.9 | Mutual exclusion implementation- version 4. (Part 1 of 2.)

```

33 } // end Thread T1
34
35 Thread T2:
36
37 void main()
38 {
39     while ( !done )
40     {
41         t2WantsToEnter = true; // enterMutualExclusion
42
43         while ( t1WantsToEnter ) // enterMutualExclusion
44         {
45             t2WantsToEnter = false; // enterMutualExclusion
46
47             // wait for small, random amount of time
48
49             t2WantsToEnter = true;
50         } // end while
51
52         // critical section code
53
54         t2WantsToEnter = false; // exitMutualExclusion
55
56         // code outside critical section
57
58     } // end outer while
59
60 } // end Thread T2

```

Figure 5.9 | Mutual exclusion implementation- version 4. (Part 2 of 2.)

The fourth version guarantees mutual exclusion and prevents deadlock, but allows for another potentially devastating problem to develop, namely, **indefinite postponement**. Because we cannot make any assumptions about the relative speeds of asynchronous concurrent threads, we must consider all possible execution sequences. The threads could, for example, proceed in tandem. Each thread can set its flag to true (line 14), then make the `while` test (line 16), then enter the body of the `while` loop, then set its flag to false (line 18), then wait for a random amount of time (line 20), then set its flag to true (line 22), then repeat the sequence beginning with the `while` test. As the threads do this, the tested conditions (lines 18 and 45) will remain true. Of course, such an execution sequence would occur with low probability—but nevertheless it could occur. If a system using this type of mutual exclusion were controlling a space flight, a heart pacemaker, or an air traffic control system, the possibility of indefinite postponement and subsequent system failure could put people's lives at risk. Therefore, version four, also, is an unacceptable solution to the mutual exclusion problem.

Dekker's Algorithm (A Proper Solution)

Figure 5.10 illustrates **Dekker's Algorithm**—a correct, two-thread mutual exclusion solution implemented purely in software with no special-purpose hardware instructions. Dekker's Algorithm still uses a flag to indicate a thread's desire to enter its critical section, but it also incorporates the concept of a "favored thread" that will enter the critical section in the case of a tie (i.e., when each thread simultaneously wishes to enter its critical section).

```

1  System:
2
3  int favoredThread = 1;
4  boolean t1WantsToEnter = false;
5  boolean t2WantsToEnter = false;
6
7  startThreads(); // initialize and launch both threads
8
9  Thread T1:
10
11 void main()
12 {
13     while ( !done )
14     {
15         t1WantsToEnter = true;
16
17         while ( t2WantsToEnter )
18         {
19             if ( favoredThread == 2 )
20             {
21                 t1WantsToEnter = false;
22                 while ( favoredThread == 2 ); // busy wait
23                 t1WantsToEnter = true;
24             } // end if
25
26         } // end while
27
28         // critical section code
29
30         favoredThread = 2;
31         t1WantsToEnter = false;
32
33         // code outside critical section
34
35     } // end outer while
36
37 } // end Thread T1

```

Figure 5.10 | Dekker's Algorithm for mutual exclusion. (Part 1 of 2.)

```

38
39 Thread T2:
40
41 void main()
42 {
43     while ( !done )
44     {
45         t2WantsToEnter = true;
46
47         while ( t1WantsToEnter )
48         {
49             if ( favoredThread == 1 )
50             {
51                 t2WantsToEnter = false;
52                 while ( favoredThread == 1 ); // busy wait
53                 t2WantsToEnter = true;
54             } // end if
55
56         } // end while
57
58         // critical section code
59
60         favoredThread = 1;
61         t2WantsToEnter = false;
62
63         // code outside critical section
64
65     } // end outer while
66
67 } // end Thread T2

```

Figure 5.10 | Dekker's Algorithm for mutual exclusion. (Part 2 of 2.)

Let us examine how Dekker's Algorithm eliminates the possibility of the indefinite postponement experienced in version 4. First note that in this algorithm, the `enterMutualExclusion` primitive is implemented by lines 15-26 and 45-56; the `exitMutualExclusion` primitive is implemented by lines 30-31 and 60-61. T_1 indicates its desire to enter its critical section by setting its flag to true (line 15). The thread then proceeds to the while test (line 17) and determines whether T_2 also wants to enter its critical section. If T_2 's flag is set to false, there is no contention between threads attempting to enter their critical sections, so T_1 skips the body of the while loop and enters its critical section (line 28).

Suppose, however, that when T_1 performs the while test (line 17), it discovers that T_2 's flag is set to true. In this case, there is contention between threads attempting to enter their respective critical sections. Thread T_1 enters the body of its while loop, where the thread examines the value of variable `favoredThread`, which is used to resolve such conflicts (line 19). If thread T_1 is the favored thread,

then T_1 skips the body of the `if` and repeatedly executes the `while` test, waiting for T_2 to set `t2WantsToEnter` to false, which, as we will see, it must eventually do. If T_1 determines that T_2 is the favored thread (line 19), T_1 is forced into the body of the `if` statement, where T_1 sets `t1WantsToEnter` to false (line 21), then loops inside the ensuing `while` as long as T_2 remains the favored thread (line 22). By setting `t1WantsToEnter` to false, T_1 allows T_2 to enter its own critical section.

Eventually, T_2 exits its critical section and executes its mutual exclusion exit code (lines 60-61). These statements set `favoredThread` back to T_1 and set `t2WantsToEnter` to false. T_1 may now pass the inner `while` test (line 22) and set `t1WantsToEnter` to true (line 23). T_1 then executes the outer `while` test (line 17). If `t2WantsToEnter` (which was recently set to false) is still false, then T_1 enters its critical section (line 28) and is guaranteed exclusive access. If, however, T_2 has quickly tried to reenter its own critical section, then `t2WantsToEnter` will be set to true, and T_1 is once again forced into the body of the outer `while` (line 19).

This time, however, T_1 is the favored thread. T_1 therefore skips the body of the `if` and repeatedly executes the outer `while` test (line 17) until T_2 sets `t2WantsToEnter` to false, allowing T_1 to enter its critical section.

Dekker's Algorithm guarantees mutual exclusion while preventing deadlock and indefinite postponement. However, the proof of this statement is not immediately apparent, due to the complex nature of mutual exclusion. For example, consider the following interesting possibility. As T_1 exits the inner busy wait loop (line 22), the system might preempt T_1 before it sets `t1WantsToEnter` to true. This would allow T_2 to loop around and attempt to reenter its own critical section. T_2 would then set `t2WantsToEnter` to true and reenter its critical section. When T_1 eventually resumes execution, it sets `t1WantsToEnter` to true. Because it is T_1 's turn (and because `t1WantsToEnter` is now set to true), if T_2 tries to reenter (line 47), it must set `t2WantsToEnter` to false and enter its inner busy wait (line 52). T_1 now will be able to enter its critical section. Thus, this circumstance, which may seem complicated at first, does not result in indefinite postponement. A rigorous proof that the algorithm never results in indefinite postponement is a more complicated task (see the Anecdote, *Why Should You Believe That Your Software Is Functioning Correctly?*), and one which the reader will find in the literature.¹⁶

Self Review

1. Identify the key problem in each of our first four attempts at implementing mutual exclusion.
2. (T/F) If the algorithm presented in Fig. 5.9 were modified such that the threads waited for different amounts of time (e.g., T_1 waits from 0 to 0.2 seconds while T_2 waits from 0.5 to 0.7 seconds), this algorithm would not suffer from indefinite postponement.

Ans: 1) Version 1: Required T_1 to go first and demanded strict alternation. Version 2: Violated mutual exclusion; both threads could enter their critical sections at once. Version 3: Enabled the two threads to deadlock so that neither would ever enter its critical section. Version 4: Allowed the (albeit remote) possibility of the two threads indefinitely postponing one

another (if they operated in tandem). 2) False. The point is that no assumptions can be made about the relative speeds of asynchronous concurrent threads. Even though the random time intervals might indeed be different, we cannot assume how fast the rest of the algorithm will run.

5.4.2 Peterson's Algorithm

The development of Dekker's Algorithm in the previous section introduces some subtle problems that arise due to concurrency and asynchronism in multiprogrammed systems. For many years, this algorithm represented the state of the practice in busy-wait solutions for enforcing mutual exclusion. In 1981, G. L. Peterson published a simpler algorithm for enforcing two-process mutual exclusion with busy waiting (Fig. 5.11).¹⁷

To illustrate the correctness of Peterson's Algorithm, let us examine the algorithm as executed by thread T_1 . Before entering its critical section, T_1 indicates that it wishes to do so by setting `t1WantsToEnter` to true (line 15). To avoid indefinite postponement, T_1 sets `favoredThread` to 2 (line 16), allowing T_2 to enter its critical section. T_1 then busy waits while `t2WantsToEnter` is true and `favoredThread` is 2. If either condition becomes false, it is safe for T_1 to enter its critical section (line 20).



Anecdote

Why Should You Believe That Your Software Is Functioning Correctly?

Dijkstra was always thought-provoking. Here is one of his best: Suppose you have software that determines whether a very large number is prime or not. If the

software determines that the number is not prime, it prints out the factors of the number, which you can multiply together to see if the product is indeed the original number. But what if the software says a number is prime? Why should you believe it?

Lesson to operating systems designers: This is one of the most profound points about computing that you may ever ponder. We put remarkable trust in computers in our personal lives and in our careers. But how can we be sure that complex software really does what it is supposed to do? How can we certify that software is indeed correct, especially when we know that we cannot possibly exhaustively test complex software? The answer may lie in the research field called "proving program correctness." This field still has many important unsolved problems—only small, relatively trivial programs have been proven correct to date. It is a wonderful topic for Ph.D. candidates in computer science.

After executing the instructions in its critical section, T_1 sets `t1WantsToEnter` to false (line 22) to indicate that it has completed its critical section.

Now let us consider how preemption affects the behavior of thread T_1 . If there is no contention for the critical section when thread T_1 performs its mutual exclusion entry code, `t2WantsToEnter` is false when thread T_1 executes line 18: thread T_1 enters its critical section (line 20). Consider the case in which thread T_1 is preempted immediately following entry into its critical section. Because thread T_2 must set `avoredThread` to 1 (line 37), the test that T_2 performs in line 39 will cause T_2 to busy wait until thread T_1 has exited its critical section and set `t1WantsToEnter` to false.

However, if `t2WantsToEnter` is true when thread T_1 attempts to enter its critical section (line 18), then T_2 must have been preempted while attempting to enter its critical section. One possibility is that T_2 was preempted while executing code in its critical section, meaning `avoredThread` = 2 and `t2WantsToEnter` = true. In this case, T_1 must simply busy wait at line 18 until T_2 completes its critical section and sets `t2WantsToEnter` to false, thereby causing the while condition to fail in line 18 and allowing T_1 to proceed.

If thread T_1 discovers that both `t2WantsToEnter` is true and `avoredThread` to 2 in line 18, T_1 will busy wait in its while loop (line 18), because it had set `avoredThread` to 2 immediately before executing line 18. Thread T_1 will wait until T_2 regains the processor and sets `avoredThread` to 1 (line 37). At this point, T_2 must busy wait because `t1WantsToEnter` is true and `avoredThread` is 1. When thread T_1 regains control of a processor, it performs the test in its while loop and enters its critical section (line 20).

If thread T_1 discovers that `t2WantsToEnter` is true and `avoredThread` to 1 in line 18, T_1 may safely enter its critical section, because `t1WantsToEnter` is true and `avoredThread` is 1; thread T_2 must busy wait until T_1 completes executing its critical section and sets `t1WantsToEnter` to false.

We will now provide a formal proof that Peterson's Algorithm guarantees mutual exclusion. We do so by proving that T_2 cannot execute while T_1 is inside its critical section. Note that the algorithm is unchanged if every instance of the characters 1 and 2 is swapped. Consequently, if we prove that T_2 cannot execute while T_1 is inside its critical section, we have also proven that T_1 cannot execute while T_2 is inside its critical section.

```

1  System:
2
3  int avoredThread = 1;
4  boolean t1WantsToEnter = false;
5  boolean t2WantsToEnter = false;
6
7  startThreads(); // initialize and launch both threads

```

Figure 5.11 | Peterson's Algorithm for mutual exclusion. (Part 1 of 2.)

```

8
9 Thread! v
10
11 void main()
12 {
13     while ( !done )
14     {
15         t1WantsToEnter = true;
16         favoredThread = 2;
17
18         while ( t2WantsToEnter && favoredThread == 2 );
19
20         // critical section code
21
22         t1WantsToEnter = false;
23
24         // code outside critical section
25
26     } // end while
27 } // end Thread T1
28
29 Thread T2:
30
31 void main()
32 {
33     while ( !done )
34     {
35         t2WantsToEnter = true;
36         favoredThread = 1;
37
38         while ( t1WantsToEnter && favoredThread == 1 );
39
40         // critical section code
41
42         t2WantsToEnter = false;
43
44         // code outside critical section
45
46     } // end while
47 } // end Thread T2
48
49 } // end Thread T2

```

Figure 5.11 | Peterson's Algorithm for mutual exclusion. (Part 2 of 2.)

To begin the proof, we observe that the value of `favoredThread` is set immediately before a thread executes its while loop. Furthermore, each thread sets the value in `favoredThread` such that the other thread is favored if both

threads wish to enter their critical sections. Also, `t1WantsToEnter` is controlled exclusively by thread T_1 ; similarly, `t2WantsToEnter` is modified exclusively by thread T_2 . Now assume that T_1 is the only thread executing inside its critical section—this implies that `t1WantsToEnter` = true and either `favoredThread` = 1 or `t2WantsToEnter` = false.

For T_2 to enter its critical section (line 39), either

- `t1WantsToEnter` must be false,
- `favoredThread` must be 2, or
- both `t1WantsToEnter` is false and `favoredThread` is 2.

This proof will assume that T_2 has successfully entered its critical section while T_1 executes inside its critical section and show that Peterson's Algorithm guarantees mutual exclusion because this cannot occur (i.e., proof by contradiction). In the first case (`t1WantsToEnter` is false), T_2 has entered its critical section because `t1WantsToEnter` is false and `favoredThread` is either 1 or 2. However, as described earlier in this section, `t1WantsToEnter` must be true if thread T_1 is also executing inside its critical section—the only time `t1WantsToEnter` is false is after T_1 has exited its critical section (line 22) and before it attempts to reenter it (line 15). Thus thread T_2 could not have entered its critical section because `t1WantsToEnter` was false. This also implies that thread T_2 could not have entered its critical section given the third condition (both `t1WantsToEnter` is false and `favoredThread` is 2).

We now must show by contradiction that T_2 cannot enter its critical section because `favoredThread` cannot be set to 2 while T_1 executes inside its critical section. To enter its critical section, T_2 must have set `t2WantsToEnter` to true (line 36), then `favoredThread` to 1 (line 37) before exiting the while loop in line 39. Therefore, if thread T_2 executes in its critical section while thread T_1 executes in its critical section, the value of `favoredThread` must have changed after thread T_2 executed line 37. Note that the only time `favoredThread` is set to 2 is when T_1 executes line 16 of its mutual exclusion entry code, requiring that T_1 exit its critical section. Note that thread T_2 has already set `t2WantsToEnter` to true in line 36. Consequently, when thread T_1 attempts to enter its critical section in line 18, it will busy wait, because both `t2WantsToEnter` is true and `favoredThread` is 2. Although T_2 may enter its critical section, now T_1 must wait until T_2 exits its critical section. This contradicts the statement that T_2 and T_1 are both executing inside their critical sections. Because we have shown by contradiction that T_2 and T_1 cannot concurrently execute inside their critical sections, we have proven that Peterson's Algorithm ensures mutual exclusion.

Deadlock and indefinite postponement are impossible in Peterson's Algorithm as long as no thread terminates unexpectedly. [Note: As always, we mean that deadlock and indefinite postponement could not be caused by the mutual exclusion entry and exit code. They nevertheless could occur if the threads misbehaved in their critical sections.] For deadlock to occur, both T_1 and T_2 must concurrently

busy wait in their while loops. This will not occur, because `favoriteThread` is either 1 or 2 and is not modified during the while loop, meaning that the while test will always fail for one thread, allowing it to enter its critical section. For indefinite postponement to occur, one thread would have to be able to continually complete and reenter its critical section while the other thread busy waited. Because each thread sets the value of `favoriteThread` to the number corresponding to the other thread before entering the while loop, Peterson's Algorithm ensures that the two threads will alternate execution of their critical sections, meaning that indefinite postponement cannot occur.

Self Review

1. What major similarity is there between Dekker's Algorithm and Peterson's Algorithm?
2. What variable is necessary to prevent indefinite postponement?

Ans: 1) They both have the same three global variables. 2) Removing the `favoriteThread` variable would allow for the possibility of indefinite postponement.

5.4.3 N-Thread Mutual Exclusion: Lamport's Bakery Algorithm

Dijkstra was the first to present a software implementation of n -thread mutual exclusion primitives.¹⁸ Knuth responded with a solution that eliminated the possibility of indefinite postponement in Dijkstra's algorithm, but still allowed a process to experience a (potentially) lengthy delay.¹⁹ This generated a series of efforts to find algorithms with shorter delays. Eisenberg and McGuire presented a solution guaranteeing that a process will enter its critical section within $n - 1$ tries.²⁰ Lamport developed a solution that is particularly applicable to networks of computers (see the Biographical Note, Leslie Lamport).²¹ The algorithm, which we discuss in depth in this section, uses a "take-a-ticket" system like those employed in busy bakeries, and has been dubbed **Lamport's Bakery Algorithm**. Burns et al. offer a solution to n -thread mutual exclusion that uses a single shared variable.²² Carvalho and Roucairol discuss enforcing mutual exclusion in computer networks.²³

Lamport was the first to introduce an algorithm that allows threads to enter critical sections quickly when access to the critical section is not contested (which is often the case).²⁴ Such **fast mutual exclusion algorithms** tend to suffer from inefficiency when the critical section is indeed a point of contention. Anderson and Kim present an algorithm that allows for fast entry into a critical section in the absence of contention and for good performance under contention.²⁵

Many of the early solutions to the n -thread mutual exclusion problem are difficult to understand because they require a large number of shared variables and complicated loops that determine if a thread can enter its critical section. Lamport's algorithm provides a simpler solution that borrows from a common real-world scenario—waiting for service at a bakery. Moreover, Lamport's algorithm does not require that any operation occur atomically.

Lamport's algorithm is modeled on a bakery in which one employee services customer requests for baked goods; this employee can service exactly one customer

at a time. If only one customer is present, the transaction is simple: the customer requests baked goods, the employee retrieves the items, the customer pays for the food and exits the bakery. However, when multiple customers concurrently request service, the employee must determine in what order to serve the customers. Many bakeries serve customers in first-come-first-served order by requiring that they take a numbered ticket from a ticket dispenser as they enter the bakery. The tickets are dispensed in ascending order (i.e., if the current ticket contains a value n , the next



Biographical Note

Leslie Lamport

Leslie Lamport received his Ph.D. in Mathematics from Brandeis University in 1972.²⁶ He has worked for the Massachusetts Computer Associates, SRI International, Digital Equipment Corporation/Compaq and is currently at Microsoft Research.²⁷ He believes that his development of a method to structure logical proofs in a hierarchical fashion that is easier to read will be his most memorable work,²⁸ but he is also well known for several computer science contributions in the area of mutual exclusion of processes that share data.

One of Lamport's major contributions was the bakery algorithm for synchronizing processes using first-come-first-served priority (see Section 5.4.3, N-Thread Mutual Exclusion: Lamport's Bakery Algorithm). Important features of the bakery algorithm are that it eliminates indefinite postponement and is resilient to processes

that terminate prematurely. This simple algorithm mimics the synchronization used for customers at a bakery.²⁹ Each process that hits the entry gate to its critical section gets a number which is one higher than the previous process's number, just as each new customer to a bakery takes a new ticket with an ordering number on it. As a process exits its critical section, the next number is "called," and the process holding that number is allowed to proceed.³⁰

Lamport produced numerous papers on methods for writing formal specifications for concurrent programs, and on proving program correctness. As part of his work on proofs, he also designed a logic system called the Temporal Logic of Actions (TLA), which can be used to verify both circuit and software logic.^{31, 32}

Some of Lamport's mutual exclusion work demonstrated that mutual exclusion can be attained

without any atomic statements, which require specific hardware support.³³ He designed at least four algorithms of varying complexity and degrees of fault tolerance.³⁴ He also invented the first mutual exclusion algorithm that runs in short, constant time if no process is currently executing in its critical section.³⁵

Lamport's most famous paper, "Time, Clocks, and the Ordering of Events in a Distributed System," describes a logical clock mechanism that is used to determine the order of events' occurrences, and a method using these clocks to synchronize distributed processes.^{36, 37, 38}

Aside from his work in mutual exclusion, Lamport also developed the LaTeX typesetting system in 1985, based on Donald Knuth's TeX typesetting language.³⁹

ticket contains the value $n + 1$). After each transaction, the employee serves the customer possessing the ticket with the lowest numerical value, which ensures that customers are served in first-come-first-served order.

Figure 5.12 presents an implementation of Lamport's algorithm for n threads. In Lamport's algorithm, each thread represents a customer that must "take a ticket" to determine when the thread can enter its critical section. When a thread possesses a ticket with the lowest numerical value, it can enter its critical section. Mutual exclusion is enforced by resetting the thread's ticket value when it exits its critical section. Unlike a real-world ticket dispenser, Lamport's algorithm allows multiple threads to obtain the same ticket number. As we will see, Lamport's algorithm includes a tie-breaking mechanism to ensure that only one thread can execute in its critical section at a time.

```

1  System:
2
3  // array that records which threads are taking a ticket
4  boolean choosing[n];
5
6  // value of the ticket for each thread initialized to 0
7  int ticket[n];
8
9  startThreads(); // initialize and launch all threads
10
11 Thread  $T_x$ :
12
13 void main()
14 {
15     x = threadNumber(); // store current thread number
16
17     while ( !done )
18     {
19         // take a ticket
20         choosing[x] = true; // begin ticket selection process
21         ticket[x] = maxValue( ticket ) + 1;
22         choosing[x] = false; // end ticket selection process
23
24         // wait for number to be called by comparing current
25         // ticket value to other thread's ticket value
26         for ( int i = 0 ; i < n ; i++)
27         {
28             if ( i == x )
29             {
30                 continue; // no need to check own ticket
31             } // end if
32

```

Figure 5.12 | Lamport's Bakery Algorithm. (Part 1 of 2.)

```

33      // busy wait while thread[i] is choosing
34      while ( choosing[i] != false );
35
36      // busy wait until current ticket value is lowest
37      while ( ticket[i] != 0 && ticket[i] < ticket[x] );
38
39      // tie-breaker code favors smaller thread number
40      if ( ticket[i] == ticket[x] && i < x )
41
42          // loop until thread[i] leaves its critical section
43          while ( ticket[i] != 0 ); // busy wait
44      } // end for
45
46      // critical section code
47
48      ticket[x] = 0; // exitMutualExclusion
49
50      // code outside critical section
51
52  } // end while
53
54 } // end Thread TX

```

Figure 5.12 | Lamport's Bakery Algorithm. (Part 2 of 2.)

Lines 3-7 declare two arrays that are shared among all n threads participating in mutual exclusion. The boolean array `choosing` (line 4) is of size n ; if thread T_x is currently selecting a ticket value, then `choosing[x]` is true. Otherwise, `choosing[x]` is false. The integer array `ticket` (line 7) contains values corresponding to each thread's ticket. Similar to `choosing`, thread T_x 's ticket value is contained in `ticket[x]`. In this example, the initial value for each thread's ticket is zero.

Lines 13-54 represent the code executed by thread T_x , one of the n threads attempting to execute their critical sections. Each thread participating in the bakery algorithm executes the same `enterMutualExclusion` (lines 19-44) and `exitMutualExclusion` (line 48) constructs. When a thread is started, it executes line 15, which stores an integer value in variable `x` to uniquely identify the thread. The thread uses this value to determine its corresponding entries in the `choosing` and `ticket` arrays.

Each thread "takes a ticket" by executing the code in lines 19-22. Line 20 indicates that the current thread is attempting to take a ticket by setting `choosing[x]` to true. As we will soon see, this step is necessary to ensure that mutual exclusion is enforced if multiple threads concurrently determine their ticket values. The thread calls method `maxValue` (line 21), which returns the largest value in the integer array `ticket`. The thread then adds one to that value and stores it as its ticket value, `ticket[x]` (line 21). Note that if there are numerous threads in the system, method `maxValue` can take substantial time to execute, increasing the likelihood that the

thread that calls `maxValue` will be preempted before the method completes. After the thread has assigned its ticket value to `ticket[x]` (line 21), the thread sets `choosing[x]` to false (line 22), indicating that it is no longer selecting a ticket value. Note that when a thread exits its critical section (line 48), the ticket value is set to zero, meaning that a thread's ticket value is nonzero only if it wishes to enter its critical section.

Unlike a real-world bakery, in which an employee calls each customer's number in turn for service, the bakery algorithm requires that each thread determine when it can enter its critical section (lines 24-44). Before entering its critical section, a thread must execute the for loop (lines 26-44) that determines the state of all threads in the system. If thread T_i , the thread to examine, and thread T_x , the thread executing the for statement, are identical (line 28), T_x executes the continue statement (line 30), which skips the remaining statements in the body of the for loop and proceeds directly to increment i in line 26.

Otherwise, T_x determines if T_i is choosing a ticket value (line 34). If T_x does not wait until T_i has chosen its ticket before entering its critical section, mutual exclusion can be violated. To understand why, let us first examine the other two conditions that are tested in each iteration of the for loop.

Line 37 tests if the current thread possesses a ticket value that is less than or equal to the ticket value for the thread it is examining. This condition is analogous to a real-world bakery—each thread must wait until it possesses the lowest nonzero ticket value.

Unlike a real-world bakery, however, two or more threads in the system may obtain the same ticket value. For example, consider a thread T_a that is preempted after method `maxValue` returns and before the thread assigns a new value to `ticket[a]` in line 21. If the next thread that executes calls `maxValue`, the method will return the same value as it did to T_a . As a result, any of the threads could obtain the same ticket value. Therefore, in the case of a tie, line 40 indicates that the thread with the lowest unique identifier proceeds first.

Let us return to line 34 to examine how mutual exclusion is violated if thread T_x does not wait if T_i is selecting a ticket value. For example, consider what happens if thread T_a is preempted after returning from method `maxValue` but before adding one to the ticket value (line 21). Assume for this example that `maxValue` returns the value 215. After T_a is preempted, several other threads execute their mutual exclusion entry code. Consider two threads, T_b and T_c , that execute after T_a is preempted and complete the ticket selection code (lines 19-22), leaving thread T_b with ticket value 216 (note that T_a 's ticket value is currently 0) and T_c with ticket value 217. It is possible for T_b to complete its mutual exclusion entry code (lines 19-14), execute its critical section (line 46) and exit its critical section (line 48) before T_a regains control of a processor. It is likewise possible for T_c to enter its critical section before T_a regains control of a processor.

If T_c is preempted while executing code in its critical section and T_a regains control of a processor, T_a completes the instruction in line 21 by setting its ticket

value to 216, which is lower than that of T_c . Thus, T_a could execute its mutual exclusion entry code and enter its critical section before T_c exits its critical section, thereby violating mutual exclusion. For this reason, T_x must wait until T_i has completed its ticket selection (line 34) before comparing ticket values in lines 36-43.

When T_x has passed all of the tests for each thread (lines 28-43), T_x is guaranteed exclusive access to its critical section. When T_x exits its critical section, it sets its ticket value to 0 (line 48) to indicate that it is no longer executing in its critical section nor is it attempting to enter its critical section.

In addition to being one of the simplest n -thread mutual exclusion algorithms, Lamport's bakery algorithm exhibits several interesting properties. For example, it does not require its instructions to be executed atomically. Recall from Section 5.3, Implementing Mutual Exclusion Primitives, that we required instructions to execute atomically. This was necessary because both Dekker's and Peterson's algorithms require multiple threads to modify a shared variable to control access to their critical sections. If each thread could read and modify this variable simultaneously on different processors, the threads might read inconsistent values of their shared variables. This might allow both threads to enter their critical sections simultaneously, violating mutual exclusion. Although many architectures provide a small set of atomic instructions (see Section 5.5, Hardware Solutions to the Mutual Exclusion Problem), it is rare to find a multiprocessor system that provides hardware to prevent threads from reading and writing data simultaneously.

Lamport's Bakery Algorithm provides an elegant solution to mutual exclusion on multiprocessor systems, because each thread is assigned its own set of variables that control access to its critical section. Although all threads in the system share the arrays `choosing` and `ticket`, thread T_x is the only thread that can modify values in `choosing[x]` and `ticket[x]`. This prevents threads from reading inconsistent data, because the variables a thread examines while performing its mutual exclusion entry code cannot be simultaneously modified by another thread.

Another interesting property of the bakery algorithm is that threads that are waiting to enter their critical sections are admitted in first-come-first-served (FCFS) order unless multiple threads possess the same ticket value. Finally, Lamport's algorithm can continue to enforce mutual exclusion even if one or more threads fail, provided that the system sets each failed thread's value in the `choosing` array to false and each thread's value in the `ticket` array to 0. Given this final provision, Lamport's Bakery Algorithm cannot suffer from deadlock or indefinite postponement, a property that is particularly important in multiprocessor and distributed systems, where the failure of a hardware device such as a processor does not necessarily result in system failure.

Self Review

1. Describe why n -thread mutual exclusion might be difficult in a distributed or networked system.
2. What would happen if the system did not perform housekeeping tasks such as setting the ticket values to zero and `choosing` values to false for terminated threads?

3. Suppose multiple threads obtain the same ticket value. In the sample code provided, the thread with the lowest unique identifier enters its critical section first. Does it matter in what order these threads enter their critical sections?

Ans: 1) There is latency between a computer's sending a message and the receiving computer's actually receiving it, meaning that mutual exclusion algorithms must account for delays between when a thread modifies a shared variable and when another thread attempts to enter its critical section. 2) The whole system could suffer from indefinite postponement. Suppose a thread terminated while choosing a ticket number, and therefore choosing for that thread would be set to true forever. Other threads wishing to enter mutual exclusion would wait forever for the terminated thread to set the choosing array entry to false. 3) Unless the programmer wishes to assign priorities to threads that possess the same ticket value, the order in which threads with identical ticket values enter their critical section does not matter. Lines 34 and 37 of Fig. 5.12 ensure that no thread with a higher or lower ticket value can enter its critical section before the threads with identical ticket values. The system need only ensure that each of the threads with the same ticket value eventually enters its critical section.

5.5 Hardware Solutions to the Mutual Exclusion Problem

In the preceding examples the software solutions to the mutual exclusion problem that make few assumptions about the system's instruction set and hardware capabilities. As discussed in Chapter 2, Hardware and Software Concepts, hardware designers tend to implement mechanisms previously handled by software to improve performance and reduce development time. This section presents several mechanisms provided in hardware to help solve the problem of mutual exclusion.

5.5.1 Disabling Interrupts

The reason mutual exclusion primitives are needed in a uniprocessor system is largely that preemption allows multiple threads to access shared data asynchronously, which can result in program errors. Threads are typically preempted by interrupts such as the interrupting clock (to signal quantum expiration). Therefore, a simple way to enforce mutual exclusion is to disable (or mask) interrupts. Unfortunately, the disabling of interrupts places limitations on what software can do inside a critical section. For example, a thread that enters an infinite loop in its critical section after disabling interrupts will never yield its processor. On a uniprocessor system, the operating system could no longer use timer interrupts to gain control of the processor, meaning that the system will hang. In real-time systems, such as an air traffic control system, such a result could put people's lives at risk.

Disabling interrupts is not a viable solution for mutual exclusion in a multiprocessor system. After all, its purpose is to ensure that preemption will not occur. However, in a multiprocessor system, two threads can execute at the same time, each on a different processor. If these threads are unsynchronized, disabling interrupts alone does not prevent them from simultaneously executing inside their critical sections. Therefore, simply disabling interrupts on either processor (or both) is insufficient to enforce mutual exclusion. In general, operating system designers avoid disabling interrupts to provide mutual exclusion. However, there are a lim-

222 Asynchronous Concurrent Execution

ited set of solutions in which it is optimal for the kernel to disable interrupts for trusted code whose execution requires a short period of time. See the Linux and Windows XP case studies in Chapters 20 and 21, respectively, for examples of how current operating systems enforce mutual exclusion by disabling interrupts.

Self Review

1. (T/F) If a thread enters an infinite loop after disabling interrupts on a multiprocessor system, the operating system can no longer execute.
2. Why should a thread avoid requesting blocking I/O in a critical section in a uniprocessor system while interrupts are disabled?

Ans: 1) False. The operating system can execute on any processor on which interrupts are not disabled. The thread in the infinite loop can be aborted or restarted, but any data it shares with other threads may be left in an inconsistent state, which may cause program errors. 2) When a thread requests blocking I/O, the operating system places that thread in the *blocked* state until it receives an I/O completion event. Since such events are generated by hardware interrupts, but the operating system would never receive this signal while interrupts remained disabled. As a result, the thread will remain waiting in the *blocked* state for an event it will never receive. This is an example of deadlock, which is further discussed in Chapter 7, Deadlock and Indefinite Postponement.

5.5.2 Test-and-Set Instruction

Disabling interrupts is rarely a practical solution to the synchronization problem, so other techniques include the use of special hardware instructions. Recall from our previous examples that shared data can become corrupted because the system may preempt a thread after it has read the value at a memory location, but before the thread can write a new value to the location. The **test-and-set** instruction enables a thread to perform this operation **atomically** (i.e., indivisibly).^{40, 41} Such operations are also described as atomic **read-modify-write (RMW) memory operations**, because the processor reads a value from memory, modifies its value in its registers and writes the modified value to memory without interruption.⁴²

The previous examples of software mutual exclusion required that a thread read a variable to determine that no other thread is executing a critical section, then set a variable, known as a **lock**, to indicate that the thread is executing its critical section. The difficulty in guaranteeing mutually exclusive access to critical sections in software was that a thread could be preempted between testing the availability of a critical section and setting a lock to keep other threads from entering their critical sections. The `testAndSet` instruction eliminates the possibility of preemption occurring during this interval.

The instruction

```
testAndSet( a, b )
```

works as follows. First the instruction reads the value of `b`, which may be either true or false. Then, the value is copied into `a`, and the instruction sets the value of `b` to true.

Figure 5.13 shows how a thread can employ `testAndSet` to enforce mutual exclusion. The global boolean variable, `occupied`, is true if either thread is in its critical section. Thread T_1 decides to enter its critical section based on its local boolean variable, `p1MustWait`. If `p1MustWait` is true, T_1 must wait; otherwise, the thread may enter its critical section. Thread T_1 initially sets variable `p1MustWait` to true. Then the thread repeatedly calls `testAndSet` on `p1MustWait` and the global variable `occupied`. If T_2 is not in its critical section, the value of `occupied` is false. In this case, the `testAndSet` assigns false to variable `p1MustWait` and sets `occupied` to true. The while fails at this point and T_1 enters its critical section. Because the indivisible hardware instruction set `occupied` to true, T_2 will be unable to enter its critical section until T_1 has reset `occupied` to false.

```

1  System:
2
3  boolean occupied = false;
4
5  startThreads(); // initialize and launch both threads
6
7  Thread  $T_1$ :
8
9  void main()
10 {
11     boolean p1MustWait = true;
12
13     while ( !done )
14     {
15         while ( p1MustWait )
16         {
17             testAndSet( p1MustWait, occupied );
18         }
19
20         // critical section code
21
22         p1MustWait = true;
23         occupied = false;
24
25         // code outside critical section
26
27     } // end while
28
29 } // end Thread  $T_1$ 
30
31 Thread  $T_2$ :
32

```

Figure 5.13 | *testAndSet instruction for mutual exclusion. (Part 1 of 2.)*

```

33 void main()
34 {
35     boolean p2MustWait = true;
36
37     while ( !done )
38     {
39         while ( p2MustWait )
40         {
41             testAndSet( p2Mustwait, occupied );
42         }
43
44         // critical section code
45
46         p2MustWait = true;
47         occupied = false;
48
49         // code outside critical section
50
51     } // end while
52
53 } // end Thread T2

```

Figure 5.13 | *testAndSet instruction for mutual exclusion. (Part 2 of 2.)*

Now suppose that T_2 already is in its critical section when T_1 wants to enter. In this case, `occupied` remains true during T_1 's repeated while tests. Therefore, T_1 continues its busy waiting until T_2 eventually leaves its critical section and sets `occupied` to false. At this point, the `testAndSet` assigns `occupied`'s value to `p1MustWait`, thus allowing T_1 to enter its critical section.

Although `testAndSet` as used here does guarantee mutual exclusion, the solution may suffer from indefinite postponement. It is possible for a thread to exit the critical section and loop around to call `testAndSet` before the competing thread has a chance to execute again.

Self Review

1. Does the algorithm in Fig. 5.13 prevent indefinite postponement?
2. (T/F) The `testAndSet` instruction enforces mutual exclusion.

Ans: 1) No, the algorithm in Fig. 5.13 requires a `favoredProcess` variable as discussed in Section 5.4.1, Dekker's Algorithm, to prevent indefinite postponement. 2) False. The `testAndSet` instruction is a tool that programmers use to simplify software solutions to mutual exclusion but the instruction itself does not enforce mutual exclusion.

5.5.3 Swap Instruction

To simplify synchronization code and improve program efficiency, most architectures provide several atomic instructions. However, each architecture supports a different set of atomic operations, meaning the `testAndSet` instruction might not

be available to the application or system programmer. In this section, we demonstrate how another instruction that performs an atomic read-modify-write memory operation can provide functionality identical to the `testAndSet` instruction.

It is common for programs to exchange (or swap) values stored in two different variables (consider, for example, the Quicksort algorithm). Although the concept is simple, a successful exchange of values between two variables in most high-level programming languages requires three instructions and the creation of a temporary variable:

```
temp = a;
a = b;
b = temp;
```

Because such swapping operations are performed regularly, many architectures support a swap instruction that enables a thread to exchange the values of the two variables atomically.

The instruction

```
swap( a, b )
```

proceeds as follows. First the instruction loads the value of `b`, which may be either true or false, into a temporary register. Then, the value of `a` is copied to `b` and the value of the temporary register is copied to `a`.

Figure 5.14 shows how a thread can employ `swap` to enforce mutual exclusion. Similar to Fig. 5.13, the global boolean variable, `occupied`, is true if either thread is in its critical section. Thread T_1 decides to enter its critical section based on its local boolean variable, `p1MustWait`. Similarly, thread T_2 decides to enter its critical section based on its local variable `p2MustWait`. Note that the `swap` instruction can be used interchangeably with the `testAndSet` instruction in this algorithm. The only difference between Fig. 5.13 and Fig. 5.14 is that we have created a "fast track" to the critical section, in which control of the critical section can be obtained by the execution of one fewer instruction (i.e., by testing the loop condition after the `swap` instruction).

```
1  System:
2
3  boolean occupied = false;
4
5  startThreads(); // initialize and launch both threads
6
7  Thread  $T_1$ :
8
9  void main()
10 {
11     boolean p1MustWait = true;
12
```

Figure 5.14 | `swap` instruction for mutual exclusion. (Part 1 of 2.)

```

13     while ( !done )
14     {
15         do
16         {
17             swap( p1MustWait, occupied );
18             } while ( p1MustWait );
19
20         // critical section code
21
22         p1MustWait = true;
23         occupied = false;
24
25         // code outside critical section
26
27     } // end while
28
29 } // end Thread T1
30
31 Thread T2:
32
33 void main()
34 {
35     boolean p2MustWait = true;
36
37     while ( !done )
38     {
39         do
40         {
41             swap( p2MustWait, occupied );
42             } while ( p2MustWait );
43
44         // critical section code
45
46         p2MustWait = true;
47         occupied = false;
48
49         // code outside critical section
50
51     } // end while
52
53 } // end Thread T2

```

Figure 5.14 | swap instruction for mutual exclusion. (Part 2 of 2.)

Self Review

1. Why might it be more likely for a swap instruction to be on a system than a testAndSet?

Ans: Many algorithms require some sort of swapping, so a hardware instruction, swap, is very useful for algorithms other than just those providing mutual exclusion.

5.6 Semaphores

Another mechanism that a system can provide to implement mutual exclusion is the **semaphore**, as described by Dijkstra in his seminal paper on cooperating sequential processes (see the Biographical Note, Edsger W. Dijkstra).⁴³ A sema-



Biographical Note

Edsger W. Dijkstra

One of computer science's most famous and prolific researchers was Edsger Wybe Dijkstra. Born in 1930 in the Netherlands, he received his Ph.D. in Computing Science from the University of Amsterdam and worked as a programmer at the Mathematisch Centrum in Amsterdam. He later served as a Professor of Mathematics at the Eindhoven Institute of Technology, then as a professor of Computer Science at the University of Texas at Austin. Dijkstra died in 2002.⁴⁴

Perhaps his best-known contribution to computer science was his article that started the structured programming revolution, "GOTO Considered Harmful,"⁴⁵ a letter to the editor published in *Communications of the ACM* in March of 1968. Dijkstra claimed that the GOTO statement, unless used carefully, quickly results in "spaghetti code"—meaning the path of execution is nearly impossible to follow.⁴⁶

Dijkstra's main contributions to operating system design

were the semaphore construct (the first object invented for use in mutual exclusion algorithms) and the layered design (an improvement over the older monolithic approach, where components that provide similar functions are grouped together in layers). He implemented both concepts in his **THE multiprogramming system**—an operating system built for the Dutch EL X8 computer.⁴⁷ He later expanded the concept of a layered system to enable the construction of a more general operating system, where the enhanced layered design could be used to build operating systems for any computer.⁴⁸ In his description of the THE system, he described how semaphores could be used for synchronization. He is also known for posing the Dining Philosophers synchronization problem, which illustrates subtle difficulties in parallel programming (see Section 7.2.4, Example: Dining Philosophers).

In 1972, Dijkstra received the Turing Award—computer science's

top honor—for his work on the ALGOL programming language which introduced block structure (to define variable scopes and mark control structure bodies) and other concepts used by most modern languages.^{49, 50, 51} Yet another achievement for Dijkstra was **Dijkstra's Algorithm**. This efficient algorithm takes a weighted graph and finds the shortest paths from a given node to each of the other nodes.

Dijkstra wrote over 1300 documents, mostly in the form of his EWDs—short essays (named after his initials) that were numbered in sequence and distributed informally to the computer science community.⁵² Some were technical and serious, while others, such as "How Do We Tell Truths That Might Hurt?" were mainly humorous.⁵³ In another EWD—"The End of Computer Science?" written in 2000—he attacked the quality of today's software and argued that more computer science research is needed to improve basic design concepts.⁵⁴

phore contains a protected variable whose integer value, once initialized, can be accessed and altered by only one of two operations, *P* and *V*. [Note: *P* and *V* are short for the Dutch words, *proberen*, which means "to test," and *verhogen*, which means "to increment."] A thread calls the *P* operation (also called the wait operation) when it wants to enter its critical section and calls the *V* operation (also called the signal operation) when it wants to exit its critical section. Before a semaphore can be used for synchronization, it must be initialized. Initialization sets the value of the protected variable to indicate that no thread is executing in its critical section. It also creates a queue that stores references to threads waiting to enter their critical sections protected by that semaphore. Note that *P* and *V* are simply abstractions that encapsulate and hide the details of mutual exclusion implementations. These operations can be applied to a system with any number of cooperating threads.

5.6.1 Mutual Exclusion with Semaphores

Figure 5.15 demonstrates how mutual exclusion is enforced using a semaphore. The system initializes the semaphore occupied to 1; such semaphores are called binary semaphores. This value indicates that the critical section is available.

The program in Fig. 5.15 uses the *P* and *V* operations as the `enterMutualExclusion()` and `exitMutualExclusion()` primitives from Section 5.2.3, Mutual Exclusion Primitives. When a thread wants to enter a critical section that is protected by semaphore *S*, the thread calls `P( S )`, which operates as follows

```

1  System:
2
3  // create semaphore and initialize value to 1
4  Semaphore occupied = new Semaphore(1);
5
6  startThreads0; // initialize and launch both threads
7
8  Thread Tx:
9
10 void main()
11 {
12     while ( !done )
13     {
14         P( occupied ); // wait
15
16         // critical section code
17
18         V( occupied ); // signal
19
20         // code outside critical section
21     } // end while
22 } // Thread TX

```

Figure 5.15 | Mutual exclusion with semaphores.

```

If  $S > 0$ 
     $S = S - 1$ 
Else
    The calling thread is placed in the semaphore's queue of waiting threads

```

Because Fig. 5.15 initializes the semaphore value to 1, only one thread will be allowed into the critical section at a time. When this thread calls P , the semaphore's value is reduced to 0. When another thread calls P , that thread will be blocked.

After a thread finishes executing in its critical section, the thread calls $V(S)$. This operation proceeds as follows

```

If any threads are waiting on  $S$ 
    Resume the "next" waiting thread in the semaphore's queue
Else
     $S = S + 1$ 

```

Thus, if any threads are waiting on the semaphore, the "next" thread, which depends on the semaphore implementation, executes. Otherwise, the value of S is incremented, allowing one more thread to enter its critical section.

A proper semaphore implementation requires that P and V be indivisible operations. Also, if several threads attempt a $P(S)$ simultaneously, the implementation should guarantee that only one thread will be allowed to proceed. The others will be kept waiting, but the implementation of P and V can guarantee that threads will not suffer indefinite postponement. For example, when a thread blocks on a semaphore, the system might place that thread in a queue associated with the semaphore. When another thread calls P , the system can select one of the threads in the queue to release. We shall assume a first-in-first-out queuing discipline for threads blocked on a semaphore (to avoid indefinite postponement).

Self Review

1. What could potentially happen if a thread called the V without having called the P operation?
2. What could potentially happen if threads that blocked on a semaphore were not dequeued in a first-in-first-out order?

Ans: 1) One scenario is that the semaphore is initially 1 and no threads are waiting. The V increments the semaphore's value to 2. Now two threads using P could both enter their critical sections. **2)** A thread could suffer from indefinite postponement.

5.6.2 Thread Synchronization with Semaphores

In the previous section, we saw how a program can use a semaphore to protect access to a critical section. Semaphores also can be used to synchronize two or more concurrent threads. For example, suppose one thread, T_1 , wants to be notified about the occurrence of a particular event. Suppose some other thread, T_2 , is capable of detecting that this event has occurred. To synchronize these two threads, T_1 executes some preliminary instructions, then calls P on a semaphore that has been ini-

tialized to 0, causing T_1 to block. Eventually, T_2 executes V to signal that the event has occurred. This allows T_1 to proceed (with the semaphore still zero).

This mechanism succeeds even if T_2 detects the event and signals it with V before T_1 waits for the event by calling P . The semaphore will have been incremented from 0 to 1, so when T_1 calls P , the operation will simply decrement the semaphore from 1 to 0, and T_1 will proceed without waiting for the event.

One example of thread synchronization is the producer/consumer relationship, introduced in Section 5.2.1, Java Multithreading Case Study, Part II. Figure 5.16 shows how to implement this relationship with semaphores. Both threads share a variable called `sharedValue`. The producer generates values and assigns them to this variable, and the consumer retrieves and processes the values the producer places in this variable. Each thread may need to wait for an event to occur before it can accomplish its task. The consumer may need to wait for a value to be produced (indicated by the producer signaling semaphore `valueProduced`): the producer must wait for a previously produced value to be consumed (indicated by the consumer signaling semaphore `valueConsumed`).

```

1  System:
2  // semaphores that synchronize access to sharedValue
3  Semaphore valueProduced = new Semaphore(0);
4  Semaphore valueConsumed = new Semaphore(1);
5  int sharedValue; // variable shared by producer and consumer
6
7  startThreads(); // initialize and launch both threads
8
9  Producer Thread:
10
11 void main()
12 {
13     int nextValueProduced; // variable to store value produced
14
15     while ( !done )
16     {
17         nextValueProduced = generateTheValue(); // produce value
18         P( valueConsumed ); // wait until value is consumed
19         sharedValue = nextValueProduced; // critical section
20         V( valueProduced ); // signal that value has been produced
21     } // end while
22 } // end producer thread

```

Figure 5.16 | Producer/consumer relationship implemented with semaphores.
(Part 1 of 2.)

```

25
26 Consumer Thread:
27
28 void main()
29 {
30     int nextValue; // variable to store value consumed
31
32     while ( !done )
33     {
34         P( valueProduced ); // wait until value is produced
35         nextValueConsumed = sharedValue; // critical section
36         V( valueConsumed ); // signal that value has been consumed
37         processTheValue( nextValueConsumed ); // process the value
38
39     } // end while
40
41 } // end consumer thread

```

Figure 5.16 | *Producer/consumer relationship implemented with semaphores.*
(Part 2 of 2.)

The implementation of each thread is straightforward. The producer generates a new value (line 17), then waits on semaphore `valueConsumed` (line 18). The semaphore's value is initially 1 (line 4), so the producer assigns the newly created value `nextValueProduced` to shared variable `sharedValue` (line 19). Then the producer signals semaphore `valueProduced` (line 20). The consumer waits on this semaphore (line 34) and, when the producer signals `valueProduced` (line 20), the consumer assigns the shared variable `sharedValue` to a local variable `nextValueConsumed` (line 35). The consumer then signals semaphore `valueConsumed` (line 36), which allows the producer to create a new value, and so on. The semaphores ensure mutually exclusive access to shared variable `sharedValue` and ensure that the threads alternate so that the consumer always reads the value that the producer has just created.

Self Review

1. (T/F) A thread can be in only one semaphore's waiting queue at a time.
2. What happens if semaphore `valueProduced` is initialized to a value of 1 instead of a value of 0?

Ans: 1) True. A thread is blocked when placed in a semaphore's queue of waiting threads, meaning that thread is unable to execute code that waits on any other semaphore. **2)** The consumer could potentially consume a value before the producer produces it.

5.6.3 Counting Semaphores

A **counting semaphore** (also called a **general semaphore**) is a semaphore that is initialized to an integral value greater than zero and commonly greater than one. A

counting semaphore is particularly useful when a resource is to be allocated from a pool of identical resources. The semaphore is initialized to the number of resources in the pool. Each *P* operation decrements the semaphore by 1, indicating that another resource has been removed from the pool and is in use by a thread. Each *V* operation increments the semaphore by 1, indicating that a thread has returned a resource to the pool and the resource may be reallocated to another thread. If a thread attempts a *P* operation when the semaphore has been decremented to zero, then the thread must wait until a resource is returned to the pool by a *V* operation.

SelfReview

1. Describe how to implement a binary semaphore with a counting semaphore.
2. (T/F) The *V* operation on a counting semaphore always adds 1 to the count.

Ans: **1)** Just initialize the counting semaphore to a value of one. **2)** False. If one or more threads are waiting, *V* lets one of the threads proceed and does not increment the count.

5.6.4 Implementing Semaphores

Semaphores can be implemented in user applications and in the kernel. Given Dekker's algorithm or the availability of a `testAndSet` or `swap` machine instruction, it is straightforward to implement *P* and *V* with busy waiting. However, busy waiting wastes processor cycles that could be put to better use in a multiprogrammed system. In Chapter 3, *Process Concepts*, we studied the thread-state switching mechanisms implemented in the kernel. We noted that a thread requesting an I/O operation voluntarily blocks itself pending completion of the I/O. The blocked thread does not busy wait—it remains asleep until the system awakens it and moves it to the ready list.

Semaphore operations can be implemented in the kernel by blocking waiting threads to avoid busy waiting.⁵⁵ A semaphore is implemented as a protected variable and a queue in which threads can wait for *V* operations. When a thread attempts a *P* operation on a semaphore whose value is zero, the thread relinquishes the processor and blocks itself to await a *V* operation on the semaphore. The system places the thread in the queue of threads waiting on that semaphore. (We assume a first-in-first-out queue discipline. Other disciplines have been investigated.)⁵⁶ The system then reassigns the processor to the next ready thread. The thread in the semaphore queue eventually moves to the head of the queue. A subsequent *V* operation removes the thread from the semaphore queue and places it on the ready list.⁵⁷

Of course, threads attempting simultaneous *P* and *V* operations on a semaphore must be guaranteed exclusive access to the semaphore by the kernel. In the case of uniprocessor systems, because *P* and *V* are so short, their indivisibility can be ensured by simply disabling interrupts while *P* and *V* operations are executing. This prevents the processor from being usurped until the operation is complete (at which point

interrupts are again enabled). This should be done with care, or it could lead to poor performance or even deadlock (see the Anecdote, Unclear Requirements).

In the kernel of a multiprocessor system, one of the processors can be given the job of controlling the ready list and determining which processors run which threads.⁵⁸ Another approach to implementing a kernel for a multiprocessor system is to control access (via busy waiting) to a shared ready list.⁵⁹ A distributed operating system kernel could have one processor control the ready list, but normally each processor manages its own ready list and essentially has its own kernel.^{60, 61, 62, 63} As a thread migrates between various processors of a distributed system, control of that thread is passed from one kernel to another.

Self Review

1. What is a major benefit of implementing semaphores in the kernel?
2. Consider a semaphore allows the thread with the highest priority to proceed when *V* is called. What potential problem can this cause?

Ans: 1) Semaphores can avoid busy waiting. The kernel can suspend a thread that attempts a *P* operation when the semaphore's current value is 0, and move it back to the *ready* queue when a *V* operation is called. This could increase performance. 2) Threads in the semaphore's wait queue may be indefinitely postponed by higher-priority threads.



Anecdote

Unclear Requirements

In his early years in industry, HMD worked for a company that developed custom software solutions for a wide range of clients. One client—an actuarial firm—sent him the following requirements—
Develop a program that will print a list of the "Connecticut pension-

ers who died this year in alphabetical order." This crossed his eyes, because pensioners, of course, rarely die in alphabetical order! His client was inaccessible for a few days, so HMD created a report program that worked as he thought it should—one that

printed a alphabetized list of all the Connecticut pensioners who died that year—which turns out to be what the client really wanted. That client was much more careful when specifying the requirements for future projects.

Lesson to operating systems designers: Before you build a complex piece of software like a substantial component of an operating system, be sure that the requirements are absolutely clear. You not only need to "solve the problem right;" you need to "solve the right problem" in the first place.

Web Resources

www.ddj.com/documents/s=924/ddj9801g/9801g.htm

Describes a method for implementing mutual exclusion in Java. It describes, both generally and specifically for Java, how a programmer might implement mutual exclusion between a number of threads.

www.es.utexas.edu/users/EWD/

Contains electronic copies of manuscripts authored by E. W. Dijkstra, including his seminal paper that introduced semaphores.

www.osdata.com/topic/language/asm/coproc.htm

Describes several machine-language instructions for concurrency, such as test-and-set and swap.

developer.java.sun.com/developer/Books/performance2/chap3.pdf

Chapter from Sun's multithreaded and network programming book. It describes how to handle mutual exclusion in Java. The discussion is motivated by describing some circumstances that mutual exclusion helps avoid; then the article provides a thorough treatment of implementing mutual exclusion to avoid these unwanted circumstances.

www.teamten.com/lawrence/242.paper/242.paper.html

Provides a survey of mutual exclusion algorithms implemented by operating systems running in a multiprocessor environment.

Summary

When more than one thread exists in a system at the same time, the threads are said to be concurrent. Two concurrent threads can execute completely independently of one another, or they can execute in cooperation. Processes that operate independently of one another but must occasionally communicate and synchronize to perform cooperative tasks are said to execute asynchronously.

When one thread reads data that another thread is writing, or when one thread writes data that another thread is also writing, indeterminate results can occur. We can solve this problem by granting each thread exclusive access to the shared variable. While one thread increments the shared variable, all other threads desiring to do so will be made to wait. This is called mutual exclusion. When the executing thread finishes accessing the shared variable, the system will allow one of the waiting processes to proceed. This is called serializing access to the shared variable. In this manner, threads will not be able to access shared data simultaneously.

In a producer/consumer relationship, the producer thread generates data and stores it in a shared object, and the consumer thread reads data from the shared object. We demonstrated how logic errors can arise with unsynchronized access among multiple threads to shared data—data could be lost if the producer places new data into the shared buffer before the consumer consumes the previous data, and data could be incorrectly duplicated if the consumer consumes data again before the producer produces the next value. If this logic were part of an air traffic control application, people's lives could be at risk.

Mutual exclusion needs to be enforced only when threads access shared modifiable data—when threads are performing operations that do not conflict with one

another (i.e., reading data), the system should allow the threads to proceed concurrently. When a thread is accessing shared modifiable data, the thread is said to be in its critical section (or critical region). To prevent the kinds of errors we encountered earlier, the system should ensure that only one thread can execute the instructions in its critical section at a time. If one thread attempts to enter its critical section while another thread executes its critical section, the first thread should wait until the executing thread exits its critical section. Once a thread has exited its critical section, a waiting thread (or one of the waiting threads, if there are several), may enter and execute its critical section. If a thread in its critical section terminates, either voluntarily or involuntarily, then the operating system, in performing its termination housekeeping, must release mutual exclusion so that other threads may enter their critical sections.

We discussed the `enterMutualExclusion()` and `exitMutualExclusion()` primitives, which invoke the most fundamental operations inherent to mutual exclusion. These primitives exhibit the following properties: each machine-language instruction is executed indivisibly; no assumption is made about the relative speeds of asynchronous concurrent threads; a thread that is executing instructions outside its critical section cannot prevent other threads from entering their critical sections; and a thread cannot be indefinitely postponed from entering its critical section.

An elegant software implementation of mutual exclusion primitives was first presented by Dekker. We followed Dijkstra's development of Dekker's Algorithm, which provides mutual exclusion while addressing the problems of busy waiting, lockstep synchronization, deadlock and

indefinite postponement. We then discussed simpler and more efficient algorithms developed by G. L. Peterson and L. Lamport. Lamport's Bakery Algorithm, which is designed for distributed systems, demonstrates a software mutual exclusion algorithm for n threads that is valid in multiprocessor systems and that does not require its operations to be performed atomically.

Several hardware mechanisms have been developed to help implement mutual exclusion. A simple way to enforce mutual exclusion using hardware is to disable (or mask) interrupts. This solution benefits from simplicity; however, the disabling of interrupts can be disastrous if a thread misbehaves inside a critical section. Moreover, disabling interrupts is not a viable solution for mutual exclusion in a multiprocessor system. Other hardware techniques have been developed, including the use of special hardware instructions. The test-and-set and swap instructions enable a thread to perform atomic read-modify-write (RMW) memory operations. These instructions eliminate the possibility of preemption between the instruction that determines if a thread can enter its critical section and the instruction that sets a variable to indicate that no other thread may enter the critical section.

Key Terms

atomic operation—Operation performed without interruption.

asynchronous concurrent threads—Threads that exist simultaneously but operate independently of one another and that occasionally communicate and synchronize to perform cooperative tasks.

binary semaphore—Semaphore whose value can be no greater than one, typically used to allocate a single resource.

busy waiting—Form of waiting where a thread continuously tests a condition that will let the thread proceed eventually; while busy waiting, a thread uses processor time.

concurrent—The description of a process or thread that exists in a system simultaneously with other processes and/or threads.

consumer—Application that reads and processes data from a shared object.

consumer thread—Thread whose purpose is to read and process data from a shared object.

counting semaphore—Semaphore whose value may be greater than one, typically used to allocate resources from a pool of identical resources.

critical region—See critical section.

Another mutual exclusion mechanism is semaphores, as described by Dijkstra. A semaphore contains a protected variable whose integer value, once initialized, can be accessed and altered only by calling one of two operations, P and V . A thread calls the P operation (also called the wait operation) when it wants to enter a critical section; the thread calls the V operation (also called the signal operation) when it wants to exit the critical section. Before a semaphore can be used for synchronization, it must be initialized. Initialization sets the value of the protected variable to indicate that no thread is executing in a critical section. A counting semaphore (also called a general semaphore) may be initialized to a value greater than one. Counting semaphores are particularly useful when resources are to be allocated from a pool of identical resources. Semaphores can be implemented in user applications and in the kernel. Given Dekker's Algorithm or the availability of a testAndSet or swap machine instruction, it is straightforward to implement P and V with busy waiting. However, busy waiting wastes processor cycles that could be put to better use in a multiprogrammed system. Semaphore operations can also be implemented in the kernel to avoid busy waiting by blocking waiting threads.

critical section—Section of code that performs operations on a shared resource (e.g., writing data to a shared variable). To ensure program correctness, at most one thread can simultaneously execute in its critical section.

deadlock—State of a thread when it cannot continue execution because it is waiting for an event that will never occur.

Dekker's Algorithm—Algorithm that ensures mutual exclusion between two threads and prevents both indefinite postponement and deadlock.

Dijkstra's Algorithm—Efficient algorithm to find the shortest paths in a weighted graph.

disable interrupts—To temporarily ignore interrupts to allow a thread on a uniprocessor system to execute its critical section atomically.

fast mutual exclusion algorithm—Implementation of mutual exclusion that avoids the overhead of a thread performing multiple tests when no other thread is contending for its critical section. This first fast mutual exclusion algorithm was proposed by Lamport.

general semaphore—See counting semaphore.

indefinite postponement—Situation in which a thread waits for an event that might never occur.

236 Asynchronous Concurrent Execution

indivisible operation—See atomic operation.

Lamport's Bakery Algorithm— N -thread mutual exclusion algorithm based on a "take a ticket" system.

lockstep synchronization—Situation where asynchronous threads execute code in strict alternation.

mutual exclusion—Restriction whereby execution by a thread of its critical section precludes execution by other threads of their critical sections. Mutual exclusion is crucial to correct execution when multiple threads access shared writable data.

mutual exclusion lock—Variable that indicates if a thread is executing its critical section; if the lock indicates that a thread is in its critical section, other threads are locked out of their own critical sections.

mutual exclusion primitives—Fundamental operations that are needed to implement mutual exclusion: `enterMutualExclusion()` and `exitMutualExclusion()`.

P operation—Operation on a semaphore. If the variable in the semaphore is 0, then the P operation blocks the calling thread. If the variable is greater than 0, the operation will decrement the variable by one and allow the calling thread to proceed.

producer—Thread or process that creates and places data into a shared object.

producer thread—Thread that creates and places data into a shared object.

producer/consumer relationship—Interaction between threads that produce data (called producers) and threads that consume produced data (called consumers) that illustrates many of the intricacies of asynchronous concurrent execution.

protected variable (semaphores)—Integer variable storing the state of a semaphore that can be accessed and altered only by calling P or V on that semaphore.

read-modify-write (RMW) memory operation—Operation that atomically reads the contents of a variable, changes the contents (possibly based on what it has read) and writes the new value to memory. These operations simplify mutual exclusion algorithms by providing atomic operations.

semaphore—Mutual exclusion abstraction that uses two atomic operations (P and V) to access a protected integer variable that determines if threads may enter their critical sections.

serialize—To control access to a shared variable such that only one thread can access the variable at a time; another thread can access the variable only after the first has finished.

signal (semaphores)—Operation on a semaphore that increments the value of the semaphore's variable. If threads are sleeping on the semaphore, the signal wakes one and decrements the semaphore's value by 1.

swap instruction—Operation that exchanges the values of two variables atomically. This instruction simplifies mutual exclusion implementations by eliminating the possibility that a thread will be preempted while performing a read-modify-write memory operation.

termination housekeeping—In the case of mutual exclusion algorithms, task performed by the operating system to ensure that mutual exclusion is not violated and that threads can continue to execute if a thread terminates while executing its critical section.

test-and-set—Instruction implemented in hardware that atomically tests the value of a variable and sets the value of the variable to true. This instruction simplifies mutual exclusion implementations by eliminating the possibility that a thread will be preempted while performing a read-modify-write memory operation.

THE Multiprogramming System—First layered operating system architecture, created by Edsger Dijkstra.

V operation—Operation on a semaphore that increments the value of the semaphore's variable if there are no threads waiting on the semaphore. If threads are waiting, the V operation wakes one of these.

wait (semaphores)—If the variable in the semaphore is 0, then the operation blocks the calling thread. If the variable is greater than 0, the operation will decrement the variable by one and allow the calling thread to proceed. Wait is also called the P operation.

Exercises

5.1 Give several reasons why the study of concurrency is appropriate and important for students of operating systems.

5.2 Explain why the following statement is false: When several threads access shared information in main memory,

mutual exclusion must be enforced to prevent the production of indeterminate results.

5.3 Dekker's Algorithm, `testAndSet`, `swap`, and the semaphore operations P and V may all be used to enforce mutual

exclusion. Compare and contrast these various schemes. Consider their respective advantages and disadvantages.

5.4 When two threads simultaneously attempt `enterMutualExclusion()`, we have assumed that the "winner" is selected at random. Discuss the ramifications of this assumption. Give a better method. Discuss how such a method might be implemented on a multiprocessor system where several threads could in fact attempt `enterMutualExclusion()` at precisely the same moment.

5.5 Comment on the use of mutual exclusion primitives in Fig. 5.17.

5.6 What is the real significance of Dekker's Algorithm?

5.7 In Dekker's Algorithm (Fig. 5.10), it is possible for T_2 to leave its critical section, execute its mutual exclusion exit code, execute its mutual exclusion entry code, and ultimately reenter its critical section before T_1 gets the chance it has been waiting for to enter its own critical section. Could T_2 actually reenter its own critical section many times before T_1 got a chance? If it could, then explain precisely how this could happen and indicate if this situation is an example of indefinite postponement. If it could not happen, then explain precisely how it is prevented.

5.8 Explain how the example concurrent program that enforces mutual exclusion with `testAndSet` (Fig. 5.13) could lead to indefinite postponement. Indicate why this possibility would nevertheless be highly unlikely. Under what circumstances would it be acceptable to use this mutual exclusion technique? Under what circumstances would it be completely unacceptable?

5.9 Perform an exhaustive analysis of Dekker's Algorithm. Does it have any weaknesses? If not, explain why.

5.10 The solution for n -thread mutual exclusion presented by Eisenberg and McGuire⁶⁴ guarantees that any single process will enter its critical section within $n-1$ tries in the worst case. Should one hope for better performance with n processes?

5.11 Mutual exclusion primitives can be implemented with busy waiting or with blocking. Discuss the applicability and relative merits of each approach.

5.12 Explain in detail how binary semaphores and binary semaphore operations can be implemented in the kernel of an operating system.

5.13 Explain how the disabling and enabling of interrupts is useful in implementing mutual exclusion primitives on uniprocessor systems.

5.14 Show how to implement semaphore operations with `testAndSet`.

5.15 Some computers have a swap instruction that, like `testAndSet`, simplifies the implementation of mutual exclusion primitives. The swap instruction simply exchanges the values of two booleans and thus requires a temporary holding area; the swap instruction is executed indivisibly.

- a. Express swap as a procedure header in a high-level language.
- b. Show how your swap procedure (assuming it is executed indivisibly) may be used to implement the `enterMutualExclusion()` and `exitMutualExclusion()` primitives.

```

1  // perform instructions outside of a critical section
2
3  enterMutualExclusion();
4
5      // perform instructions inside a critical section
6
7      enterMutualExclusion();
8
9          // perform instructions inside a nested critical section
10
11      exitMutualExclusion();
12
13      // perform instructions inside a critical section
14
15  exitMutualExclusion();
16
17  // perform instructions outside of a critical section

```

Figure 5.17 | Code for Exercise 5.5.

5.16 As mentioned in the text, critical sections that reference nonintersecting sets of shared variables may indeed be executed simultaneously. Suppose the mutual exclusion primitives are each modified to include a parameter list of the particular shared variables to be referenced in a critical section.

- Comment on the use of these new mutual exclusion primitives in Fig. 5.18.
- Suppose the two threads in Fig. 5.19 operate concurrently. What are the possible outcomes?

5.17 In Dekker's Algorithm, what (if anything) would happen if the two assignment statements in the mutual exclusion exit code were reversed?

5.18 Show that Peterson's Algorithm (Fig. 5.11) is *bounded fair*⁶⁵; i.e., a thread cannot be delayed indefinitely on any delay condition that occurs with indefinite repetition. In particular,

show that any thread waiting to enter its critical section will be delayed no longer than the time it takes for the other thread to enter and leave its own critical section once.

5.19 Present a detailed analysis of Peterson's Algorithm to demonstrate that it works properly. In particular, show that deadlock cannot occur, that indefinite postponement cannot occur, and that mutual exclusion is enforced.

5.20 Show that if a system implementing Lamport's Bakery Algorithm does not perform termination housekeeping, the system could suffer from indefinite postponement.

5.21 Based on your understanding of the kernel and interrupt handling, describe how semaphore operations may be implemented on a uniprocessor system.

```

1 // perform instructions outside of a critical section
2
3 enterMutualExclusion(a);
4
5 // perform instructions inside a critical section
6
7 enterMutualExclusion(b);
8
9 // perform instructions inside a nested critical section
10
11 exitMutualExclusion(b);
12
13 // perform instructions inside a critical section
14
15 exitMutualExclusion(a);
16
17 // perform instructions outside of a critical section

```

Figure 5.18 | New mutual exclusion primitives for Exercise 5.16(a).

Thread T_1

```

1 enterMutualExclusion(a);
2
3 enterMutualExclusion(b);
4
5 exitMutualExclusion(b);
6
7 exitMutualExclusion(a);

```

Thread T_2

```

1 enterMutualExclusion(b);
2
3 enterMutualExclusion(a);
4
5 exitMutualExclusion(a);
6
7 exitMutualExclusion(b);

```

Figure 5.19 | Code for Exercise 5.16(b).

5.22 In the text we implied that busy waiting can be wasteful. Is it always wasteful? What alternatives exist? Discuss the pros and cons of busy waiting.

5.23 If many threads attempt a *P* operation, which one should be allowed to proceed? What are the key issues here? What criteria might you use to decide which thread should proceed on a uniprocessor system? What criteria might you use on a multiprocessor system?

5.24 A system supports only binary semaphores. Show that counting semaphores may be simulated on this system by using binary semaphores.

5.25 One requirement in the implementation of a *P* and *V* is that each of these operations must be executed indivisibly; i.e., once started, each operation runs to completion without interruption. Give an example of a simple situation in which, if

these operations are not executed indivisibly, mutual exclusion may not be properly enforced.

5.26 Suppose the only mutual exclusion primitive provided to user-level threads is a command that disables interrupts for the next 32 instructions, then reenables interrupts. Explain the benefits and drawbacks of this approach.

5.27 How could cooperating threads implement mutual exclusion primitives in the system mentioned in the previous exercise?

5.28 Does the code in Fig. 5.20 provide mutual exclusion? If not, show an interleaving in which mutual exclusion is not preserved?

5.29 Name one other mutual exclusion constraint the algorithm in Fig. 5.20 violates.

```

1  System:
2
3  int turn = 1;
4  boolean t1WantsToEnter = false;
5  boolean t2WantsToEnter = false;
6
7  startThreads(); // initialize and launch both threads
8
9  Thread T1:
10
11 void main()
12 {
13     while ( !done )
14     {
15         t1WantsToEnter = true;
16
17         while ( turn != 1 )
18         {
19             while ( t2WantsToEnter );
20
21             turn = 1;
22         } // end while
23
24         // critical section code
25
26         t1WantsToEnter = false;
27

```

Figure 5.20 | Algorithm for Exercise 5.28 (Part 1 of 2.)

240 Asynchronous Concurrent Execution

```
28      // code outside critical section
29    } // end outer while
30 } // end Thread T1
31
32 Thread T2:
33
34 void main()
35 {
36     while ( !done )
37     {
38         t2WantsToEnter = true;
39
40         while ( turn != 2 )
41         {
42             while ( t1WantsToEnter );
43
44             turn = 2;
45         } // end while
46
47         // critical section code
48
49         t2WantsToEnter = false;
50
51         // code outside critical section
52     } // end outer while
53 } // end Thread T2
```

Figure 5.20 | AAlgorithm for Exercise 5.28. (Part 2 of 2.)

Suggested Projects

5.30 Prepare a research paper on Lamport's "fast mutual exclusion" algorithm. How does it work? Where is it used?

5.31 Research mutual exclusion in distributed systems. What research is currently being done in that field?

5.32 Research mutual exclusion multiprocessor systems. Why might this be easier than mutual exclusion in distributed systems?

5.33 Many mutual exclusion algorithms are verified using computer programs. Research how these programs manage to verify that a certain algorithm is correct.

5.34 Dijkstra is probably best known for his shortest-path algorithm. However, he also contributed greatly in other fields of computer science. For example, we know from the text that he invented semaphores. Prepare a biography of E. W. Dijkstra and the important contributions he made to the field of computer science. See, for example, www.cs.utexas.edu/users/EWD/.

5.35 Research the Ada concurrency primitives. How is mutual exclusion guaranteed in this language?

Suggested Simulations

5.36 Implement semaphores in Java. Then use them to provide synchronization between two threads in a producer/consumer program.

Recommended Reading

The problem of process synchronization was identified early and often in the literature.^{66, 67, 68} The related problem of mutual exclusion has been studied in detail, and many resources are available that present solutions.^{69, 70, 71} Dijkstra first described the problem on mutual exclusion, identified critical sections and applied Dekker's Algorithm to solve the problem.⁷² Subsequently, Peterson⁷³ and Lamport⁷⁴ presented novel approaches to implementing software mutual exclusion, both of which simplified code by introducing a more structured

approach. Though Dijkstra first presented a solution to n -thread mutual exclusion, Knuth,⁷⁵ Eisenberg and McGuire⁷⁶, Lamport,⁷⁷ Anderson and Kim⁷⁸ and others have continued to improve on the original algorithm. Dijkstra also introduced semaphores in his seminal paper on mutual exclusion.⁷⁹ In response, researchers developed techniques to improve upon the concept of semaphores.^{80, 81, 82, 83} The bibliography for this chapter is located on our Web site at www.dentel.com/books/os3e/Bibliography.pdf.

Works Cited

- Atwood, J. W., "Concurrency in Operating Systems," *Computer*, Vol. 9, No. 10, October 1976, pp. 18-26.
- Thomas, R. H., "A Majority Consensus Approach to Concurrency Control," *ACM Transactions on Database Systems*, Vol. 4, 1979, pp. 180-209.
- Wegner, P., and S. A. Smolka, "Processes, Tasks and Monitors: A Comparative Study of Concurrent Programming Primitives," *IEEE Transactions on Software Engineering*, Vol. SE-9, No. 4, 1983, pp. 446-462.
- Chandy, K. M., and J. Misra, "Asynchronous Distributed Simulation via a Sequence of Parallel Computations," *Communications of the ACM*, Vol. 24, No. 4, April 1981.
- Schlichting, R. D., and F. B. Schneider, "Understanding and Using Asynchronous Message Passing Primitives," *Proceedings of the Symposium on Principles of Distributed Computing*, August 18-20, 1982, Ottawa, Canada, ACM, New York, pp. 141-147.
- Bernstein, A. X., "Program Analysis for Parallel Processing," *IEEE Transactions on Computers*, Vol. 15, No. 5, October 1966, pp. 757-762.
- Courtois, P. X; Heymans, E; and D. L. Parnas, "Concurrent Control with Readers and Writers," *Communications of the ACM*, Vol. 14, No. 10, October 1971, pp. 667-668.
- Lamport, L., "Concurrent Reading and Writing," *Communications of the ACM*, Vol. 20, No. 11, November 1977, pp. 806-811.
- Ricart, G., and A. K. Agrawala, "An Optimal Algorithm for Mutual Exclusion in Computer Networks," *Communications of the ACM*, Vol. 24, No. 1, January 1981, pp. 9-17.
- Jones, D. W., "An Empirical Comparison of Priority Queue and Event Set Implementations," *Communications of the ACM*, Vol. 29, No. 4, April 1986, pp. 300-311.
- Raynal, M., *Algorithms for Mutual Exclusion*, Cambridge, MA: MIT Press, 1986.
- Dijkstra, E. W., "Cooperating Sequential Processes," Technological University, Eindhoven, Netherlands, 1965, reprinted in E. Genuys, ed., *Programming Languages*, New York: Academic Press, 1968, pp. 43-112.
- Dijkstra, E. W., "Cooperating Sequential Processes," Technological University, Eindhoven, Netherlands, 1965, reprinted in E. Genuys, ed., *Programming Languages*, New York: Academic Press, 1968, pp. 43-112.
- Peterson, G. L., "Myths About the Mutual Exclusion Problem," *Information Processing Letters*, Vol. 12, No. 3, June 1981, pp. 115-116.
- Lamport, L., "A New Solution of Dijkstra's Concurrent Programming Problem," *Communications of the ACM*, Vol. 17, No. 8, August 1974, pp. 453-455.
- Dijkstra, E. W., "Cooperating Sequential Processes," Technological University, Eindhoven, Netherlands, 1965, reprinted in F. Genuys, ed., *Programming Languages*, New York: Academic Press, 1968, pp. 43-112.
- Peterson, G. L., "Myths About the Mutual Exclusion Problem," *Information Processing Letters*, Vol. 12, No. 3, June 1981, pp. 115-116.
- Dijkstra, E. W., "Solution of a Problem in Concurrent Programming Control," *Communications of the ACM*, Vol. 8, No. 5, September 1965, p. 569.
- Knuth, D., "Additional Comments on a Problem in Concurrent Programming Control," *Communications of the ACM*, Vol. 9, No. 5, May 1966, pp. 321-322.
- Eisenberg, M. A., and M. R. McGuire, "Further Comments on Dijkstra's Concurrent Programming Control Problem," *Communications of the ACM*, Vol. 15, No. 11, November 1972, p. 999.
- Lamport, L., "A New Solution to Dijkstra's Concurrent Programming Problem," *Communications of the ACM*, Vol. 17, No. 8, August 1974, pp. 453-455.
- Burns, X. E.; P. Jackson; N. A. Lynch; M. J. Fischer; and G. L. Peterson, "Data Requirements for Implementation of N -Process Mutual Exclusion Using a Single Shared Variable," *Journal of the ACM*, Vol. 29, No. 1, January 1982, pp. 183-205.
- Carvalho, O. S. E., and G. Roucairol, "On Mutual Exclusion in Computer Networks," *Communications of the ACM*, Vol. 26, No. 2, February 1983, pp. 146-147.

242 Asynchronous Concurrent Execution

24. Leslie Lamport, "A Fast Mutual Exclusion Algorithm," *ACM Transactions on Computer Systems (TOCS)*, Vol. 5, No. 1, pp.1-11, Feb. 1987.
25. Anderson, James H., and Yong-Jik Kim, "An Improved Lower Bound for the Time Complexity of Mutual Exclusion," *Proceedings of the Twentieth Annual ACM Symposium on Principles of Distributed Computing*, pp. 90-99, August 2001, Newport, Rhode Island, United States.
26. Lamport, L., "The Writings of Leslie Lamport," September 8, 2003, <research.microsoft.com/users/lamport/pubs/pubs.html>.
27. Lamport, L., "The Writings of Leslie Lamport," September 8, 2003, <research.microsoft.com/users/lamport/pubs/pubs.html>.
28. Milojicic, D., "A Discussion with Leslie Lamport," September 2002, <dsonline.computer.org/0208/f/lam_print.htm>.
29. Anderson, J., "Lamport on Mutual Exclusion: 27 Years of Planting Seeds," August 2001, <www.cs.unc.edu/~anderson/papers/lamport.pdf>. *Proceedings of the 20th Annual ACM Symposium on Principles of Distributed Computing*, pp. 3-12.
30. Lamport, L., "A New Solution of Dijkstra's Concurrent Programming Problem," *Communications of the ACM*, Vol. 17, No. 8, August 1974, pp. 453-455, <research.microsoft.com/users/lamport/pubs/bakery.pdf>.
31. Lamport, L., "TLA: The Temporal Logic of Actions," August 26, 2003, <research.microsoft.com/users/lamport/tla/tla.html>.
32. Lamport, L., et al., "Specifying and Verifying Systems with TLA+," September 2002, <research.microsoft.com/users/lamport/pubs/spec-and-verifying.pdf>.
33. Anderson, J., "Lamport on Mutual Exclusion: 27 Years of Planting Seeds," August 2001, <www.cs.unc.edu/~anderson/papers/lamport.pdf>. *Proceedings of the 20th Annual ACM Symposium on Principles of Distributed Computing*, pp. 3-12.
34. Anderson, J., "Lamport on Mutual Exclusion: 27 Years of Planting Seeds," August 2001, <www.cs.unc.edu/~anderson/papers/lamport.pdf>. *Proceedings of the 20th Annual ACM Symposium on Principles of Distributed Computing*, pp. 3-12.
35. Lamport, L. "A Fast Mutual Exclusion Algorithm," November 14, 1985, <research.microsoft.com/users/lamport/pubs/fast-mutex.pdf>.
36. Lamport, L., "The Writings of Leslie Lamport," September 8, 2003, <research.microsoft.com/users/lamport/pubs/pubs.html>.
37. Anderson, X., "Lamport on Mutual Exclusion: 27 Years of Planting Seeds," August 2001, <www.cs.unc.edu/~anderson/papers/lamport.pdf>. *Proceedings of the 20th Annual ACM Symposium on Principles of Distributed Computing*, pp. 3-12.
38. Lamport, L., "Time, Clocks, and the Ordering of Events in a Distributed System," *Communications of the ACM*, Vol. 21, No. 7, July 1978, pp.558-565, <research.microsoft.com/users/lamport/pubs/time-clocks.pdf>.
39. The LaTeX Project, "LaTeX Project home page," January 27, 2003, <www.latex-project.org/>.
40. Gilbert, Philip, and W. J. Chandler, "Interference Between Communicating Parallel Processes," *Communications of the ACM*, Vol. 15, No. 6, June 1972, p. 436.
41. Presser, Leon, "Multiprogramming Coordination," *ACM Computing Surveys (CSUR)*, Vol. 7, No. 1, January 1975, p. 38.
42. Kruskal, Clyde P.; Larry Rudolph; and Marc Snir, "Efficient Synchronization of Multiprocessors with Shared Memory," *ACM Transactions on Programming Languages and Systems (TOPLAS)*, October 1988, p. 580.
43. Dijkstra, E. W., "Cooperating Sequential Processes," Technological University, Eindhoven, Netherlands, 1965, reprinted in F. Genuys, ed., *Programming Languages*, New York: Academic Press, 1968, pp. 43-112.
44. Computer Science Department of the University of Texas at Austin, "E. W. Dijkstra Archive: Obituary," August 2002. <www.cs.utexas.edu/users/EWD/obituary.html>.
45. Dijkstra, E., "GOTO Considered Harmful," March 1968. <www.acm.org/classics/oct95/>. Reprinted from *Communications of the ACM*, Vol. 11, No. 3, March 1968, pp. 147-148.
46. Dijkstra, E., "GOTO Considered Harmful," March 1968. <www.acm.org/classics/oct95/>. Reprinted from *Communications of the ACM*, Vol. 11, No. 3, March 1968, pp. 147-148.
47. Dijkstra, E., "Appendix of The Structure of the "THE" Multiprogramming System," May 1968, <www.acm.org/classics/mar96/>. Reprinted from *Communications of the ACM*, Vol. 11, No. 5, May 1968, pp. 345-346.
48. Dijkstra, E., "Structure of an Extendable Operating System," November 1969, <www.cs.utexas.edu/users/EWD/ewd02xx/EWD275.PDF>.
49. Computer Science Department of the University of Texas at Austin, "E. W. Dijkstra Archive: Obituary," August 2002. <www.cs.utexas.edu/users/EWD/obituary.html>.
50. T Whaley, "CS 313—Edsger Wybe Dijkstra," <cs.wlu.edu/~whaley/classes/313/Turing/Crieco-Dijkstra.htm>.
51. Department of Computer and Information Science of the University of Michigan at Dearborn, "The ALGOL Programming Language," November 24, 1996, <www.engin.umd.umich.edu/CIS/course.des/cis400/algol/algol.html>.
52. Computer Science Department of the University of Texas at Austin, "E. W. Dijkstra Archive: Obituary," August 2002. <www.cs.utexas.edu/users/EWD/obituary.html>.
53. Dijkstra, E., "How Do We Tell Truths That Might Hurt?" 1975. <www.cs.utexas.edu/users/EWD/ewd04xx/EWD498.PDF>.
54. Dijkstra, E., "The End of Computing Science?" 2000. <www.cs.utexas.edu/users/EWD/ewd13xx/EWD1304.PDF>.
55. Hansen, P. Brinch, "The Nucleus of a Multiprogramming System," *Communications of the ACM*, Vol. 13, No. 4, April 1970, pp. 238-241, 250.
56. Stark, Eugene, "Semaphore Primitives and Starvation-Free Mutual Exclusion," *Journal of the Association for Computing Machinery*, Vol. 29, No. 4, October 1982, pp. 1049-1072.